preliminary: no circulation please

Graph Learning

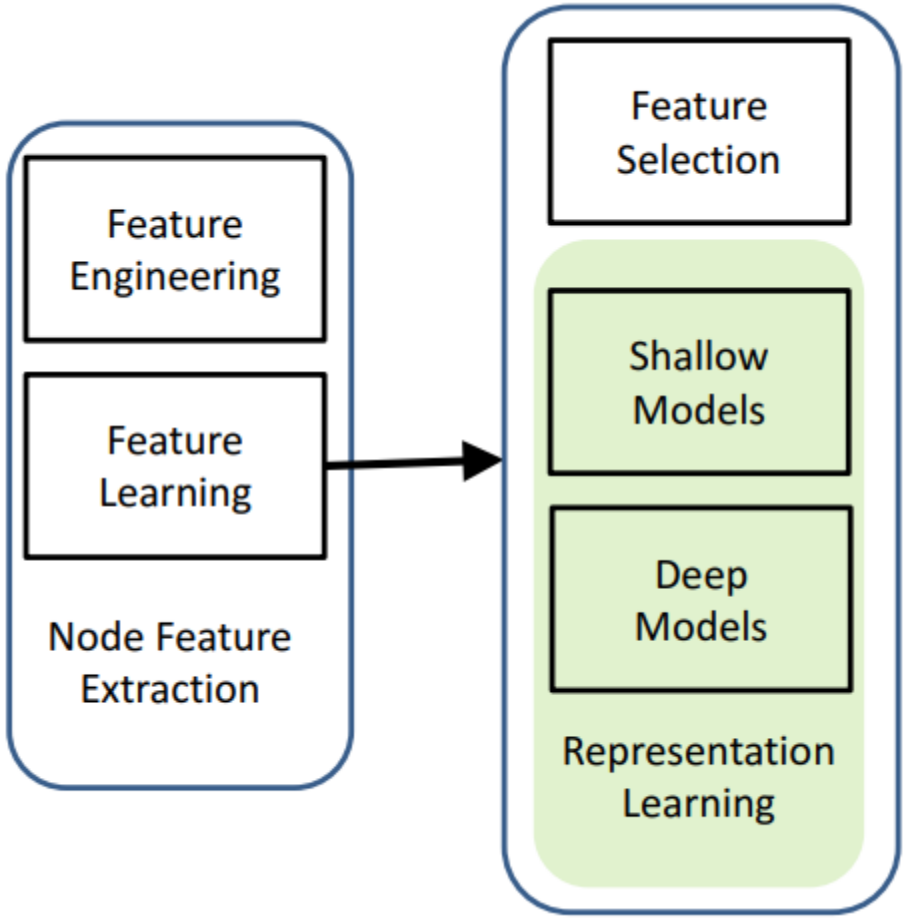
정재식
서강대학교 경제학부

Graph learning

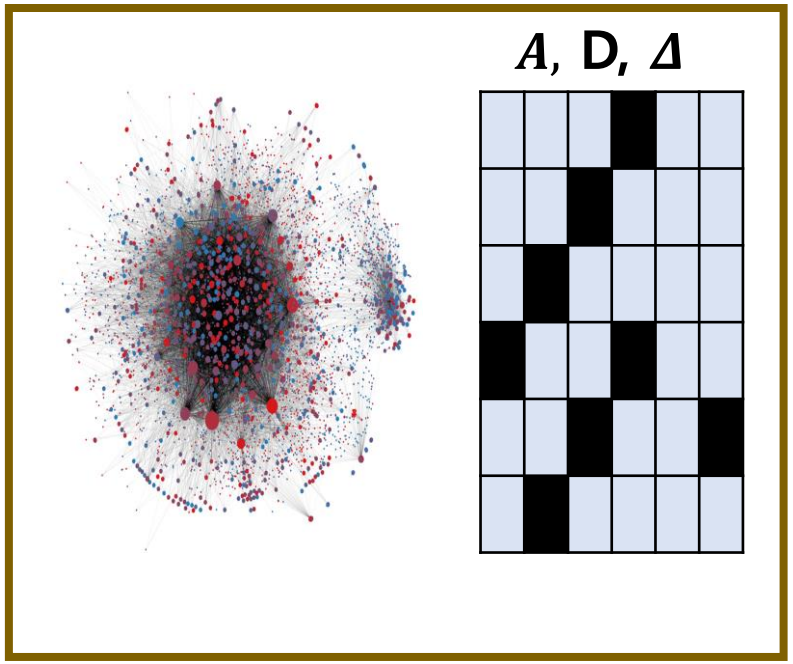
- Feature learning
  - supervised (features vs. label)
- Graph representation learning
  - spectral clustering
  - graph dimension reduction
  - matrix factorization

Feature Learning

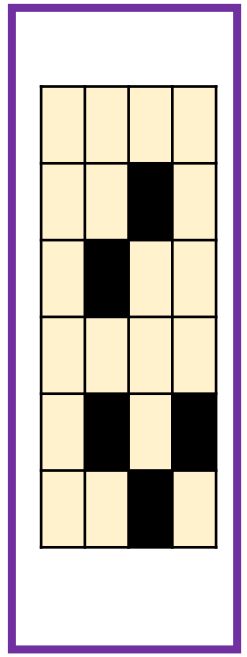
Feature learning on Graphs



downstreaming



Raw Data



structured data

$$f(\quad)$$

Learning model

- anomaly detection
- edge prediction
- clustering
- attribute prediction

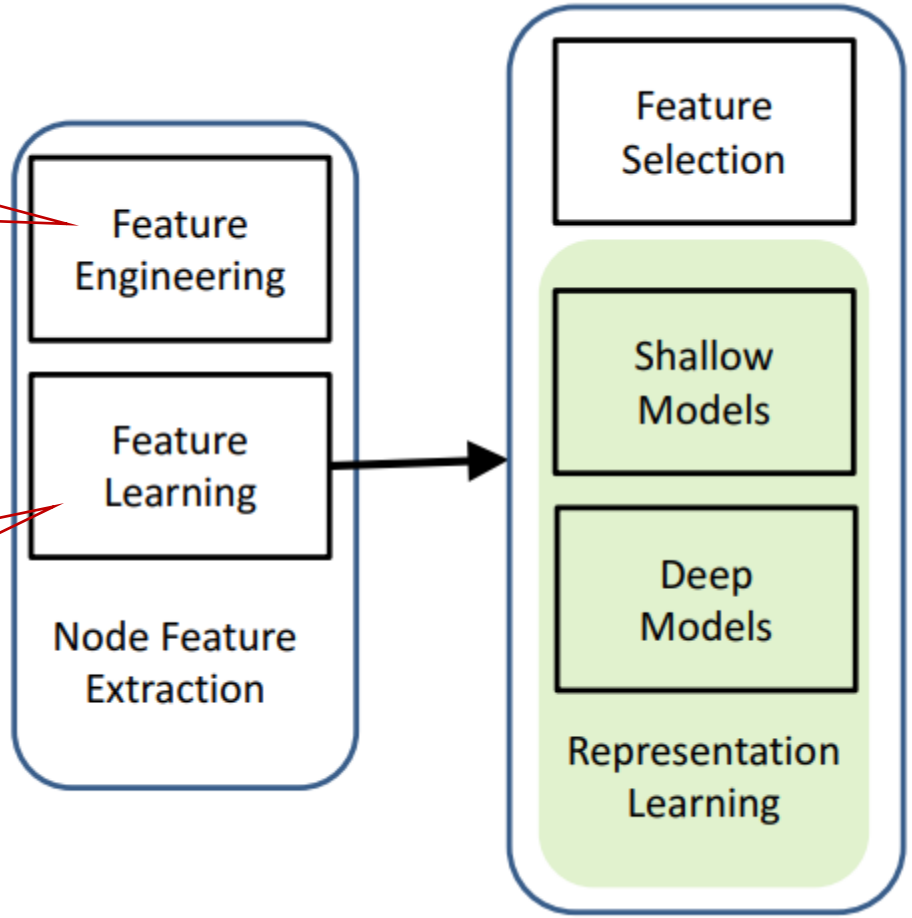
downstreams

Feature learning on Graphs

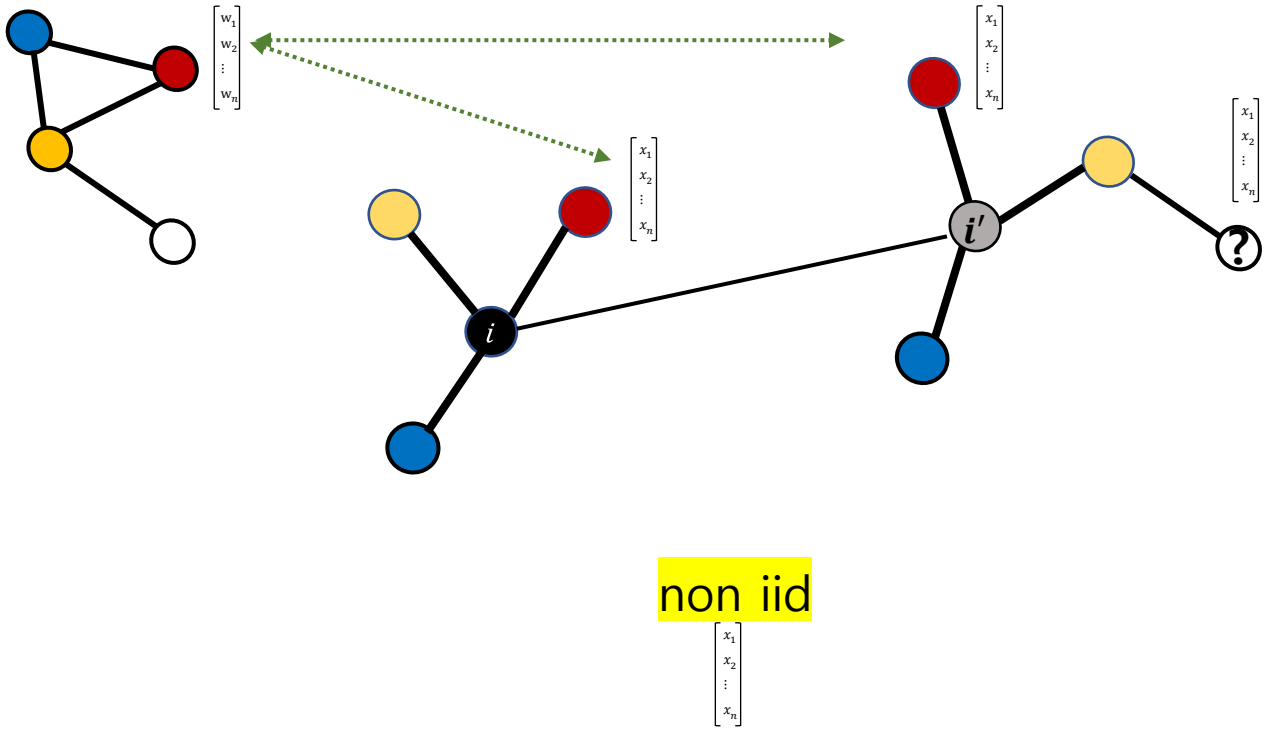
suboptimal for downstream task: node degree statistics etcs.

optimal for downstream task: extensively studied

cancer study: genes and their semantics



Feature learning: *feature selection*



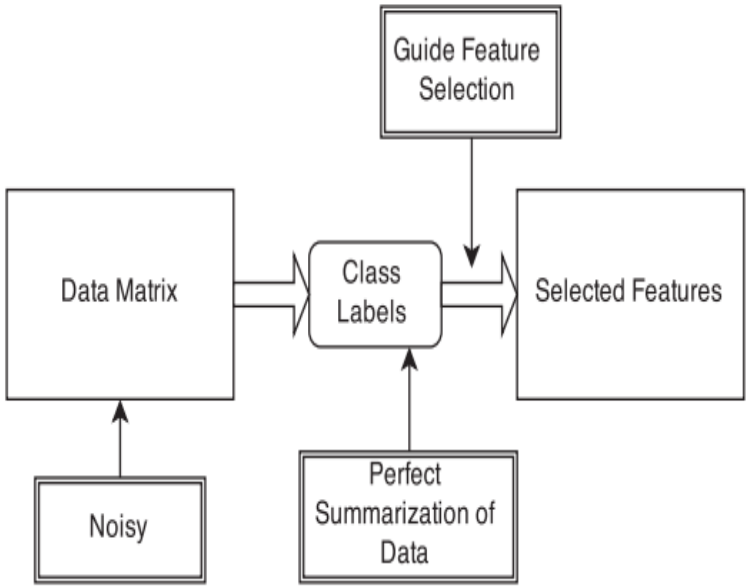
gene inducing cancerogenesis?

non iid

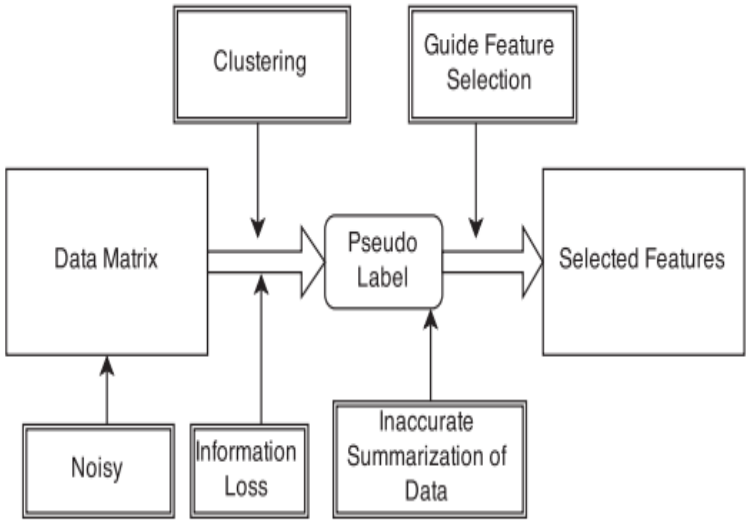
feature selection: *unsupervised setting*

- Represent information on local and global structure in a numerical way
- Node classification (using spectral clustering or non-negative matrix decomposition) and predict label with sparse features
  - spectral clustering: see related ppt
  - NMD: document-term frequency matrix decomposition
- Generative feature selection

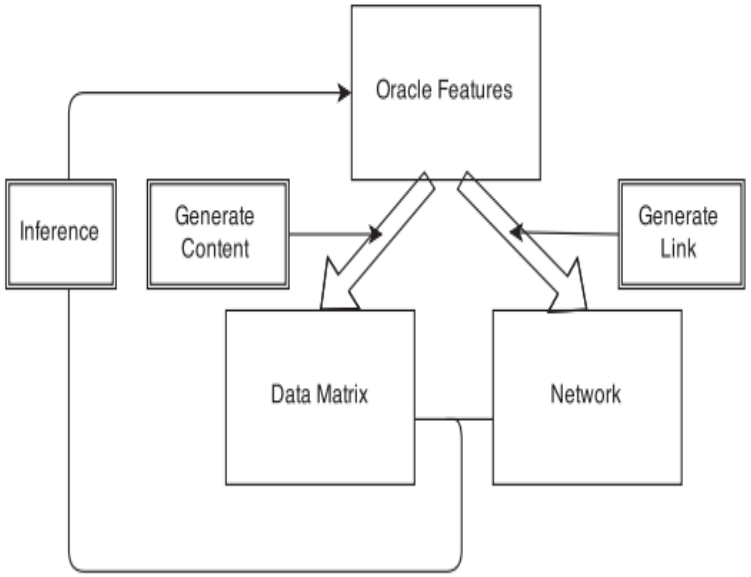
Feature selection approach



(a) Supervised feature selection



(b) Pseudo-label approach



(c) Generative Feature Selection

https://www.aaai.org/ocs/index.php/AAAI/AAAI16/paper/download/12463/11864

Unsupervised Feature Selection on Networks: A Generative View

Xiaokai Wei*, Bokai Cao* and Philip S. Yu*†
*Department of Computer Science, University of Illinois at Chicago, IL, USA
†Institute for Data Science, Tsinghua University, Beijing, China
{xwei2,caobokai,psyu}@uic.edu

issue. In this paper, we investigate the problem of unsupervised feature selection on networks. Most existing feature selection methods fail to incorporate the linkage information, and the state-of-the-art approaches usually rely on pseudo labels generated from clustering. Such cluster labels may be far from accurate and can mislead the feature selection process. To address these issues, we propose a generative point of view for unsupervised features selection on networks that can seamlessly exploit the linkage and content information in a more effective manner. We assume that the link structures and node content are generated from a succinct set of high-quality features, and we find these features through maximizing the likelihood of the generation process. Experimental results on three real-world datasets show that our approach can select more discriminative features than state-of-the-art methods.

On Exploring Semantic Meanings of Links for Embedding Social Networks

Here

ABSTRACT

There are increasing interests in learning low-dimensional and dense node representations from the network structure which is usually high-dimensional and sparse. However, most existing methods fail to consider semantic meanings of links. Different links may have different semantic meanings because the similarities between two nodes can be different, e.g., two nodes share common neighbors and two nodes share similar interests which are demonstrated in node-generated content. In this paper, the former type of links are referred to as structure-close links while the latter type are referred to as content-close links. These two types of links naturally indicate there are two types of characteristics that nodes expose in a social network. Hence, we propose to learn two representations for each node, and render each representation responsible for encoding the corresponding type of node characteristics, which is achieved by jointly embedding the network structure and inferring the type of each link. In the experiments, the proposed method is demonstrated to be more effective than five recent methods on four social networks through applications including visualization, link prediction and multi-label classification.

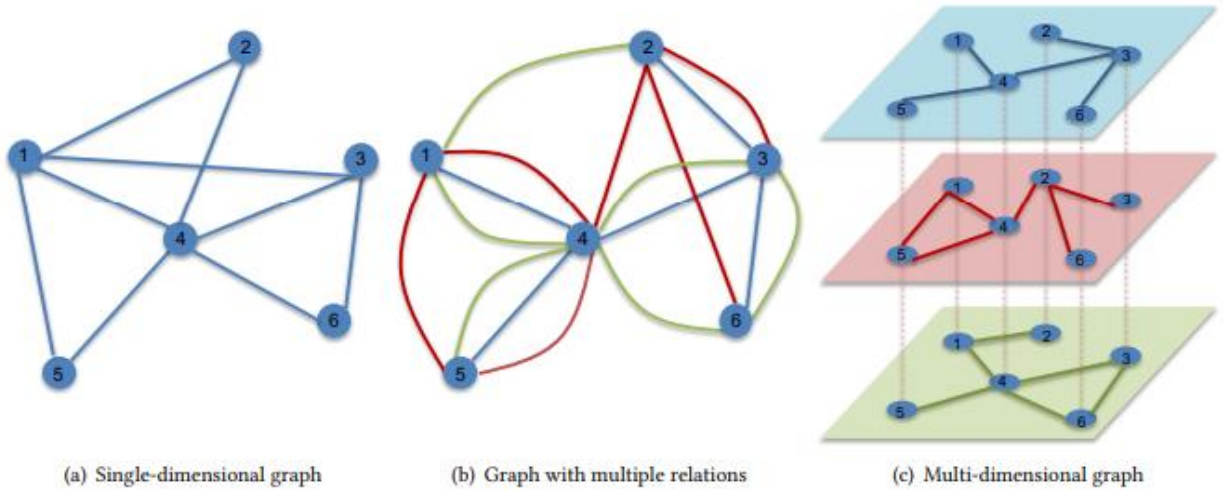
Feature selection: *unsupervised setting*

- dynamic graphs
- multi dimensional graphs
- signed graphs
- attributed graphs

Feature selection: dynamic graphs

- social media
  - messages are streaming as relationship evolves
- recommend system
  - new products and new ratings
- computational finance
  - *transaction streaming*
  - supply chains are evolving

Feature selection: multidimensional graph



(a) Single-dimensional graph (b) Graph with multiple relations (c) Multi-dimensional graph

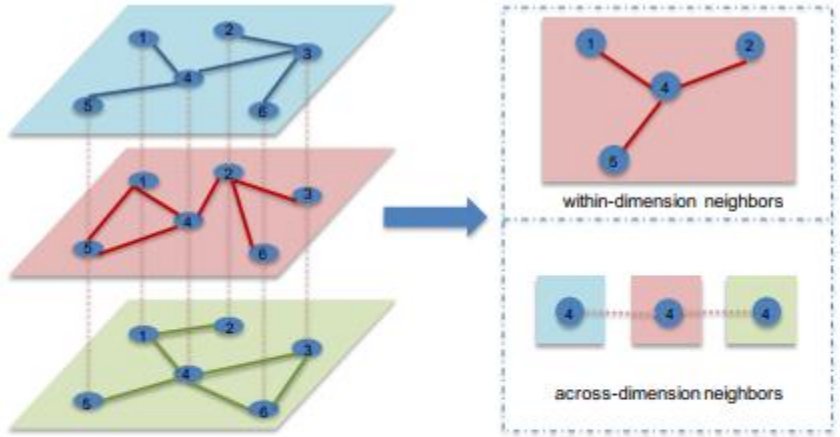


Figure 2: There are two types of neighbors in a multi-dimensional graph for a node in a given dimension. For example, for node 4 in the "red" dimension, the within-dimension neighbors are nodes 1, 2 and 5 in the "red" dimension, while its across-dimension neighbors are node 4 in the "blue" dimension and node 4 in the "green" dimension.

Feature selection: multidimensional graph

- **DBLP:** DBLP[1] is a computer science bibliography website. It holds bibliographic information on major computer science journals and proceedings. We collect all publication records from major computer science journals and conferences during 1998-2017. We then treat the co-authorship in each year as different relations and form a co-authorship multi-dimensional graph. In this multi-dimensional graph, each year forms a dimension. If two authors cooperate to publish a paper in a given year, we create a link between them in the dimension of the given year. We also assign labels to the authors according to all the papers they published. More specifically, each paper belongs to a high-level

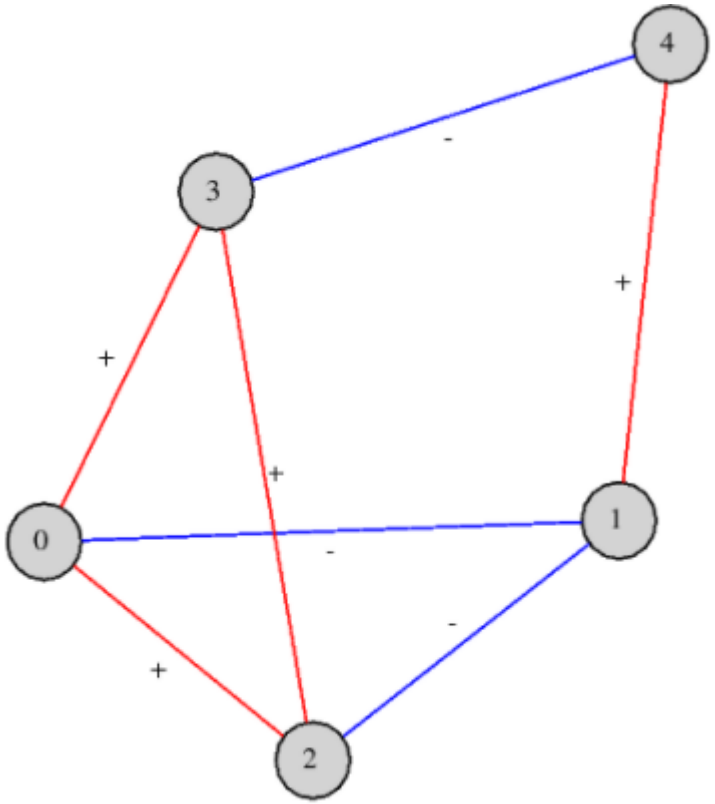
- **Epinions** Epinions[2] is a general review site, where users can write reviews for products and rate helpfulness for reviews written by other users. Users in this site can also form trust and distrust relations. We form a 5-dimensional graph based on 5 different relations between users: 1) co-review: if two users review some common products, we create a link between them in the co-review dimension; 2) helpfulness-rating: we create a link between two users in the helpfulness-rating dimension if a user rates the reviews written by the other user; 3) co-rating: if two users rate some common reviews, we create a link between them in the co-rating dimension; 4) trust: We create the trust dimension based on the trust relation between users; and 5) distrust: We create the distrust dimension based on the distrust relation between users. We also assign labels to the users according to the products they reviewed. More specifically, each product belongs to a category. We assign the category which most of the products reviewed by the user belong to as his/her label.

dynamic graph datasets

Dataset	Type	Nodes	Edges	Granularity
Social Evolution	Social network	83	376-791 Associations 2,016,339 Communications	6 mins
Github	Social network	12,328	70,640-166,565 Associations 604,649 Communications	-
HEP-TH	Citation network	1,424-7,980	2,556-21,036	Monthly
Autonomous systems	Communication network	103-6,474[5]	243-13,233	Daily
GDELT	Events knowledge graph	14,018	31.29M	15 mins
ICEWS	Events knowledge graph	12498	0.67M	Daily
YAGO	Knowledge graph	15,403	138,056	Mostly yearly
Wikidata	Knowledge graph	11,134	150,079	Yearly
Reddit	Social network	55,863	858,49	Seconds
Enron	Email network	151	50.5K	Seconds
FB-Forum	Social network	899	33.7K	Seconds
Blog	Social network	5,196	171,743	-
Cora[6]	Citation network	2708	5429	-
Flicker	Social network	1,715,256	22,613,981	-
UCI	Communication network	1,899	59,835	Seconds
Radoslaw[7]	Email network	167	82.9K	Seconds
DBLP[89]	Citation network	315,159	743,70	-
YELP	Bipartite ratings	6,569	95,361	Seconds
MovieLens-10M	Bipartite ratings	20,537	43,760	Seconds
CONTACT	Face-to-face proximity	274	28,200	Seconds
HYPERTEXT09	Face-to-face proximity	113	20,800	Seconds
Elliptic[10]	Bitcoin transactions	203,769	234,355	49 time steps

Table 2: A summary of the datasets used in dynamic (knowledge) graph publications, the type of data they contain, the number of nodes and edges they contain, and their temporal granularity.

Feature selection: Signed Graphs



See Signed graph math here

Feature selection: attributed graphs

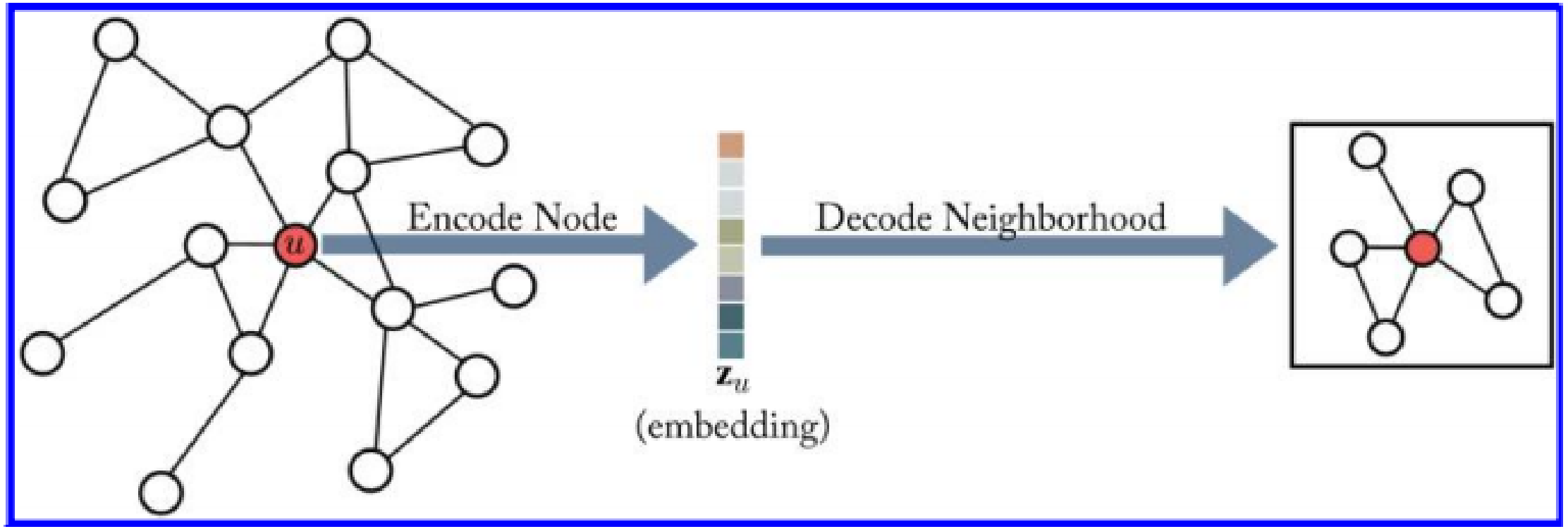
- each nodes is associated with a feature vectors and nodes are connected by edges encoding
- Examples are citation networks, web graphs, social networks
  - **citation**: each node is a document's vector of words and edges are citation links
  - **web graph**: webpage with vector of words and edges are hyperlinks
  - **sns**: user profile and edges are friendshipcomputational finance
  - **protein-protein relation**

Graph representation Learning

Graph Representation learning

- Learn representations that encode structural information about the graph *(rather than extracting hand-engineered features)*

Graph Representation learning: encoding-decoding



Graph Representation learning

- See Hamilton's book titled "Graph representation learning"
  - **auto encoder- decoder**
  - **variation auto encoder**
  - **GNN**
  - **Deep Generative model**

Graph Representation for what?

- Node classification
- Graph classification
- Relation prediction

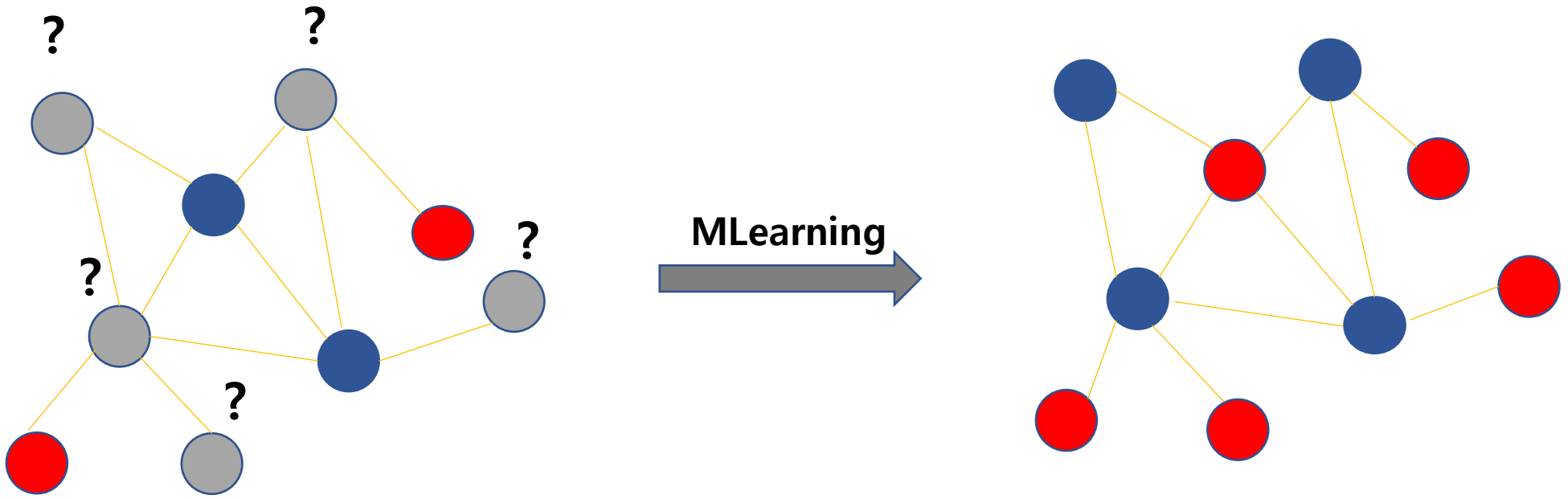
Embeddings related

- Embeddings?
  - Shallow embeddings
  - Deep embeddings
- Graph spectral
- GCN/GNN

Why Graph matter in economics/finance?

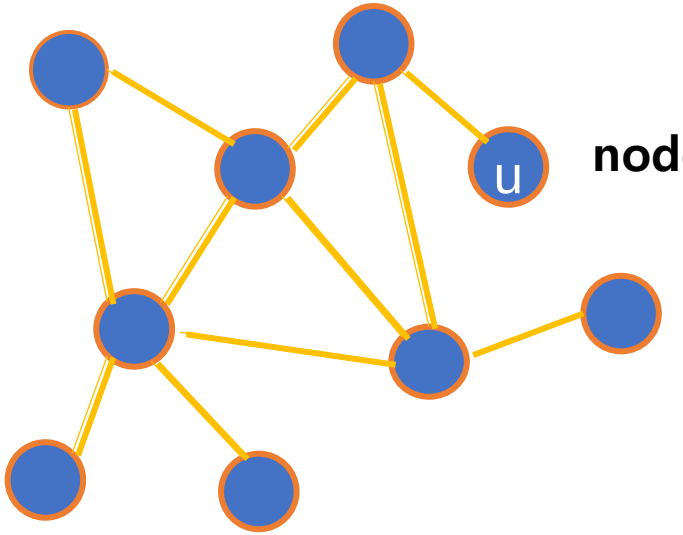
- Intuitive
- being able to deal with complex problem

learning in graph



learning in graph

network embedding in graph



node

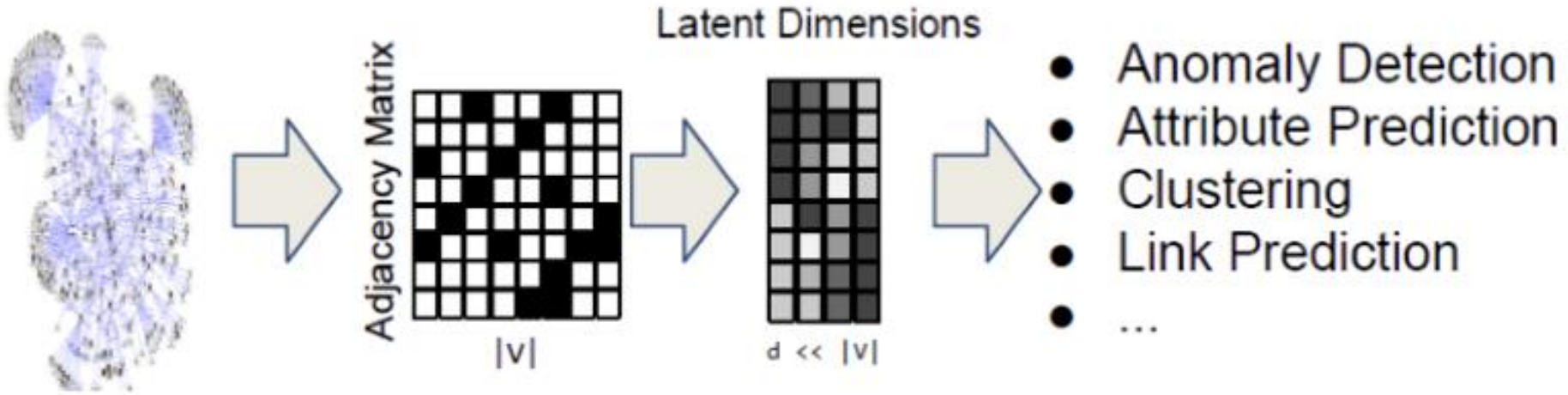
$$f: u \rightarrow \mathbb{R}^d$$

feature representation
embedding

Embeddings Overview

network embedding in graph

- embedding: mapping *each* node into a vector space
  - node similarity ⟺ vector similarity



Embedding: Matrix Factorization

- factorization: use adjacency matrix A
  - GraRep: $s(v_i, v_j \triangleq A^2_{i,j})$
  - HOPE: Jaccard neighborhood

Shallow Embedding: random walk based

- Extending skip-gram model
- DeepWalk (Perozzi et al. (2014))
- node2vec (Grover and Leskovec (2016))

Embedding method:

- Tang et al. (2015)
  - seeks to preserve first- and second-order proximity and trains the embedding via negative sampling

- Wang et al. (2016)
  - jointly uses unsupervised components to preserve second order proximity and exploit first-order proximity in its supervised components;

Deep Embedding method:

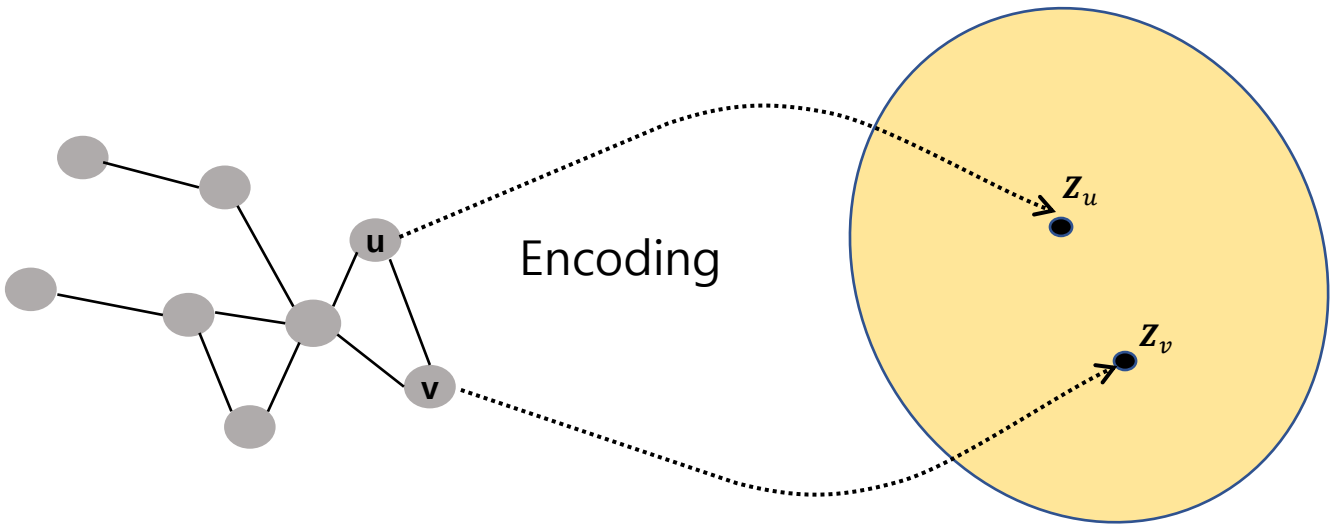
- GraphSAGE
  - Hamilton et al. (2017)): utilize node attributes and potentially node labels.

- Convolutional neural networks are also applied to graph-structured data.
  - Kipf and Welling (2017)

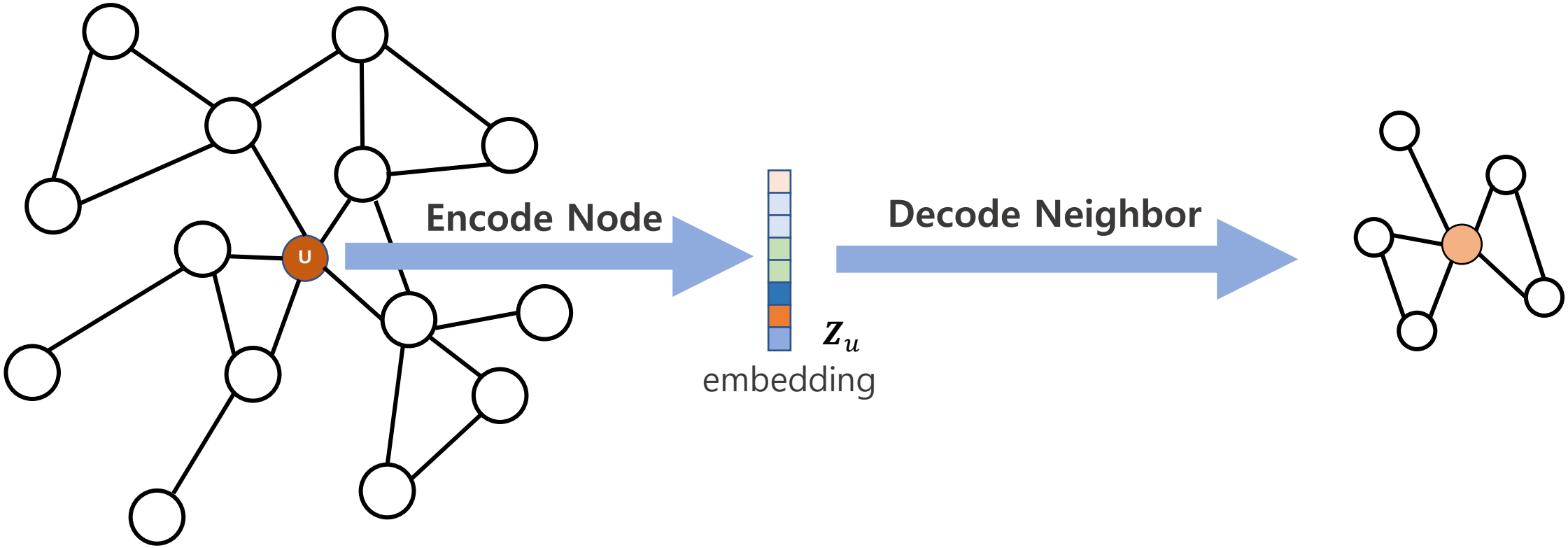
- inductive graph embedding learning
  - Hamilton et al. (2017), Velickovic et al. (2017) Bojchevski and Gunnemann ¨ (2017) Derr et al. (2018) Gao et al. (2018), Li et al. (2018), Wang et al. (2018) and Ying et al. (2018b))

What are Embeddings?

Encoding



Embedding

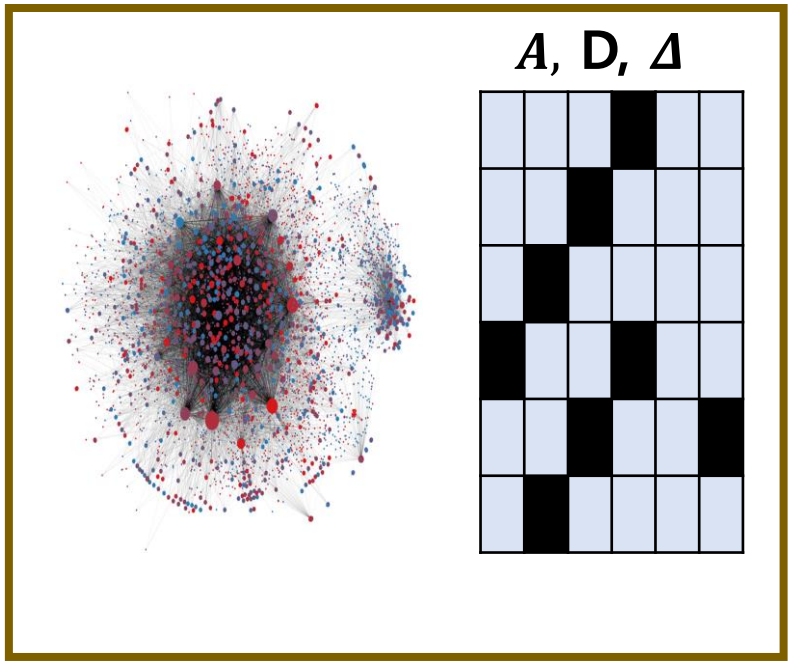


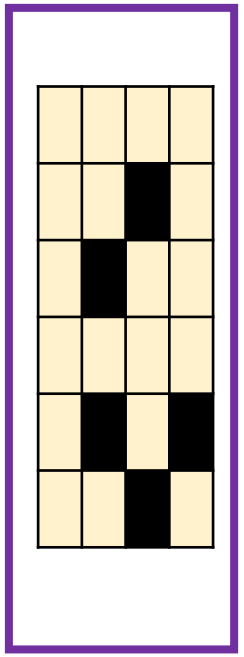
Why embeddings?

- Used for many downstream purpose
  - node classification
  - link prediction
  - graph classification
  - anomaly detection

- Downstream: to learn patterns from structured data and applying them to entities (or among entities)

downstreaming





$$f\left(\right)$$

- anomaly detection
- edge prediction
- clustering
- attribute prediction

Raw Data

structured data

Learning model

downstreams

Embedding nodes: Notation

- **Encoder function**
  - $ENC: \mathfrak{v} \rightarrow R^d$
- **Decoder function**
  - $DEC: R^d \times R^d \rightarrow R^+$
- **graph similarity ($s(\mathfrak{v}_i, \mathfrak{v}_j)$)**
  - $DEC\left(ENC(\mathfrak{v}_i), ENC(\mathfrak{v}_j)\right) = DEC(z_i, z_j) \approx s(\mathfrak{v}_i, \mathfrak{v}_j)$
- **loss function**
  - $\mathcal{L} = \sum_{(\mathfrak{v}_i, \mathfrak{v}_j) \in \mathfrak{v}} \| DEC(z_i, z_j) - s(\mathfrak{v}_i, \mathfrak{v}_j) \|$

Embeddings?: shallow

Shallow encoding

- The simplest approach is using 'embedding-lookup'

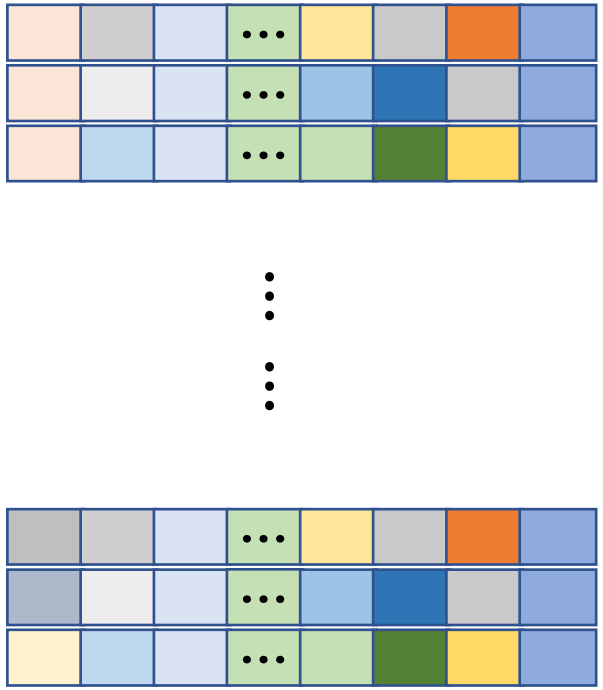
$$\mathrm{ENC}(v) = z_v = \mathrm{Z} \cdot v$$

$\mathrm{Z} \in \mathbb{R}^{|\mathcal{V}| \times d}$: each row is a node embedding

v: $\mathbb{R}^{|\mathcal{V}|}$: indicator vector (all zeros except one)

Shallow encoding

$$Z = \underset{\mathbb{R}^{|\mathcal{V}| \times d}}{} \left[\begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \end{array} \right]$$



$\mathbb{R}^{|\mathcal{V}|}$: number of node
one row per node

$\mathbb{R}^{d}$: embedding dim

ENC-DEC methods

Decoder simiarity

Method	Decoder	Similarity Measure	Loss Function
Lap Eigenmaps	$\|\|\mathbf{z}_u - \mathbf{z}_v\|\|_2^2$	General	$\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]$
Graph Factorization	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v]$	$\|\|\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
GraRep	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v], \ldots, \mathbf{A}^k[u, v]$	$\|\|\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
HOPE	$\mathbf{z}_u^\top \mathbf{z}_v$	General	$\|\|\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
DeepWalk	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p\mathcal{G}(v\|u)$	$-\mathbf{S}[u, v] \log(\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v))$
node2vec	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p\mathcal{G}(v\|u)$ (biased)	$-\mathbf{S}[u, v] \log(\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v))$

$p\mathcal{g}$: $prob. of\ vising\ v_j\ on\ a\ fixed\ length\ random\ walk\ staring\ from\ v_i$

ENC-DEC methods

node simiarity

Method	Decoder	Similarity Measure	Loss Function
Lap Eigenmaps	$\|\|\mathbf{z}_u - \mathbf{z}_v\|\|_2^2$	General	$\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]$
Graph Factorization	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v]$	$\|\|\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
GraRep	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v], \ldots, \mathbf{A}^k[u, v]$	$\|\|\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
HOPE	$\mathbf{z}_u^\top \mathbf{z}_v$	General	$\|\|\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
DeepWalk	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p\mathcal{G}(v\|u)$	$-\mathbf{S}[u, v] \log(\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v))$
node2vec	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p\mathcal{G}(v\|u)$ (biased)	$-\mathbf{S}[u, v] \log(\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v))$

$p\mathcal{G}$: $prob. of\ vising\ v_j\ on\ a\ fixed\ length\ random\ walk\ staring\ from\ v_i$

ENC-DEC methods

node
simiarity

Method	Decoder	Similarity Measure	Loss Function
Lap Eigenmaps	$\|\|\mathbf{z}_u - \mathbf{z}_v\|\|_2^2$	General	$\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]$
Graph Factorization	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v]$	$\|\|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
GraRep	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v], \ldots, \mathbf{A}^k[u, v]$	$\|\|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
HOPE	$\mathbf{z}_u^\top \mathbf{z}_v$	General	$\|\|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
DeepWalk	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p_{\mathcal{G}}(v\|u)$	$-\mathbf{S}[u, v] \log(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v))$
node2vec	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p_{\mathcal{G}}(v\|u)$ (biased)	$-\mathbf{S}[u, v] \log(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v))$

$p_{\mathcal{G}}$: $prob. of\ vising\ v_j\ on\ a\ fixed\ length\ random\ walk\ staring\ from\ v_i$

ENC-DEC methods

matrix(A or Δ) factorization

random walk

Method	Decoder	Similarity Measure	Loss Function
Lap Eigenmaps	$\|\|\mathbf{z}_u - \mathbf{z}_v\|\|_2^2$	General	$\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]$
Graph Factorization	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v]$	$\|\|\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
GraRep	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v], \ldots, \mathbf{A}^k[u, v]$	$\|\|\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
HOPE	$\mathbf{z}_u^\top \mathbf{z}_v$	General	$\|\|\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]\|\|_2^2$
DeepWalk	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in \mathcal{V}} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p\mathcal{G}(v\|u)$	$-\mathbf{S}[u, v] \log(\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v))$
node2vec	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in \mathcal{V}} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p\mathcal{G}(v\|u)$ (biased)	$-\mathbf{S}[u, v] \log(\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v))$

$p\mathcal{g}$: $prob. of\ vising\ v_j\ on\ a\ fixed\ length\ random\ walk\ staring\ from\ v_i$

Embeddings: Random Walks

Random Walk embeddings

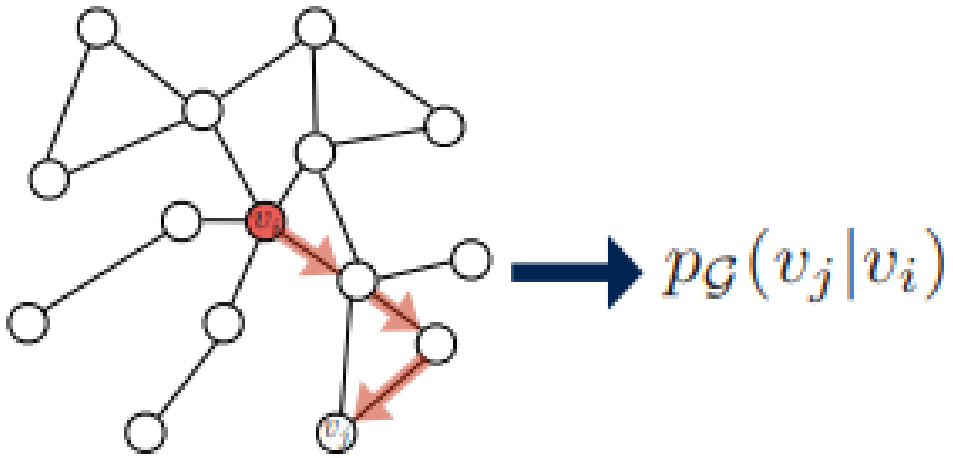
- "Unsupervised" node embeddings
  - not using node labels or features
  - "*model is optimized or trained, over set of node pairs to reconstruct pairwise similarity values only depends on graph*"

Random walk: reference

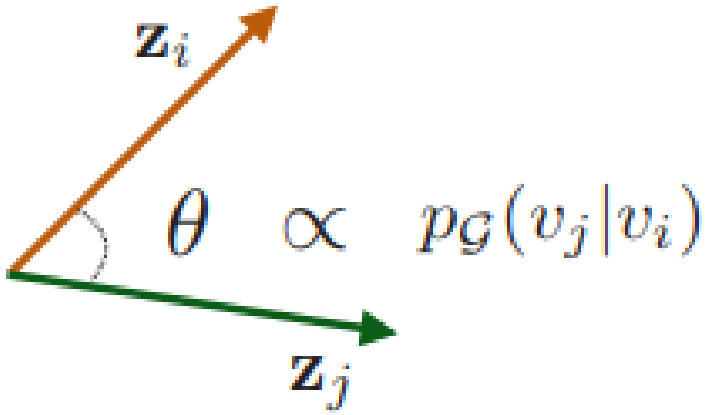
- See Stanford CS224W Machine learning with Graphs, 2021.
  - lecture 3.2: Random walk approaches for node embeddings

- Read the paper of Representation learning or Graphs

Random walk

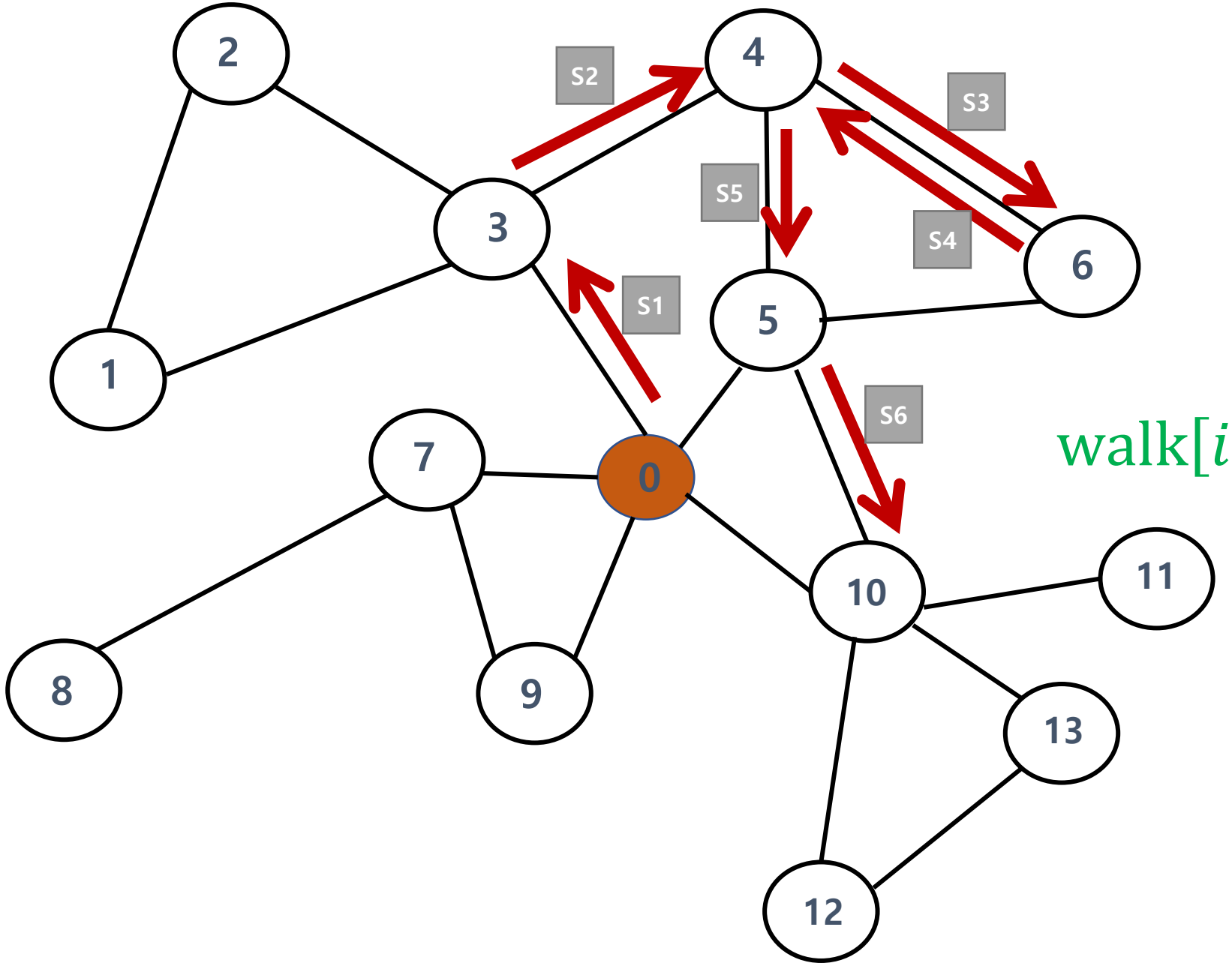


1. Run random walks to obtain co-occurrence statistics.

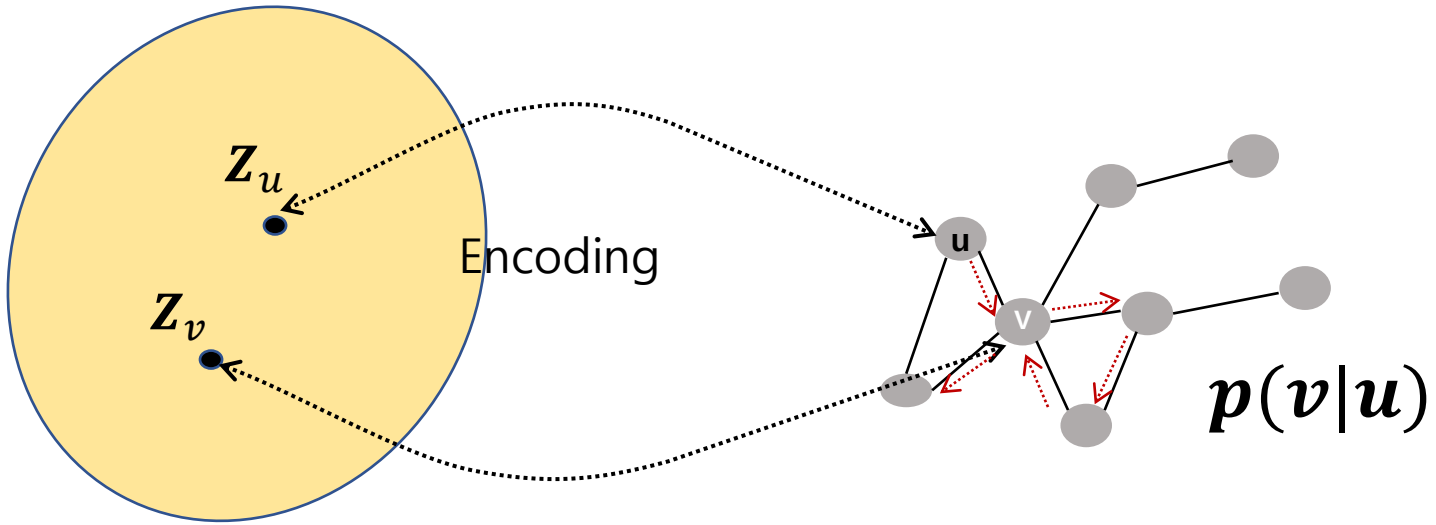


2. Optimize embeddings based on co-occurrence statistics.

Many recent successful methods that also belong to the class of shallow embedding approaches learn the node embeddings based on random walk statistics. Their key innovation is optimizing the node embeddings so that nodes have similar embeddings if they tend to co-occur on short random walks over the graph (Figure 4). Thus,



walk[i]: [0 3 4 6 4 5 6 ....]



$$Z_u^T Z_v \uparrow \quad = \quad \text{probability that u and v co-occur on random walks over the graph}$$

$$= \quad p(v|u) \uparrow$$

random walk embeddings

- Given $G = (\mathcal{V}, \mathcal{E})$
- Want to learn a mapping $z: u \rightarrow \mathbb{R}^d$
- Log-likelihood

$$\max_z \sum_{u \in \mathcal{V}} \log P(\mathcal{N}_{\mathrm{R}}(u) | z_u)$$

where $N_R(u)$ is neighborhood of node u by strategy R

- *"Given node u, want to learn feature representation of predicting nodes in* $\mathrm{N}_R(u)$*"*

random walk optimization

- fit parameters to maximize the possibility of random walk co-occurrence

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \mathrm{P}(v|z_u)$$

random walk optimization (softmax parameterization)

- softmax parameterization:
  - normalized exponential function
- fit parameters to maximize the possibility of random walk co-occurrence

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \mathrm{P}(v|z_u)$$

$$= \sum_{u \in V} \sum_{v \in N_R(u)} -\log \left(\frac{\exp(z_u^T z_v)}{\sum_{n \in V} \exp(z_u^T z_n)} \right)$$

random walk optimization (softmax parameterization)

- fit parameters to maximize the possibility of random walk co-occurrence

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \mathrm{P}(v|z_u)$$

$$= \sum_{u \in V} \sum_{v \in N_R(u)} -\log\left(\frac{\exp(z_u^T z_v)}{\sum_{n \in V} \exp(z_u^T z_n)}\right)$$

for all nodes in G

random walk optimization (softmax parameterization)

- fit parameters to maximize the possibility of random walk co-occurrence

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \mathrm{P}(v|z_u)$$

$$= \sum_{u \in V} \sum_{v \in N_R(u)} -\log\left(\frac{\exp(z_u^T z_v)}{\sum_{n \epsilon V} \exp(z_u^T z_n)}\right)$$

for all neighbor (or randomly visited nodes) for a specific node *u*]

DeepWalk vs. Node2Vec: unbiased vs. biased walks

- random walks
  - DW: unbiased random walks
    - next step is random or depenps on given weight
- N2V: biased (p, q) random walks
  - p: the likelihood of the walk immeidately revisiting a node
  - q: the likelihood of the walk revisiting a node's one-hop neighborhood

Random walk

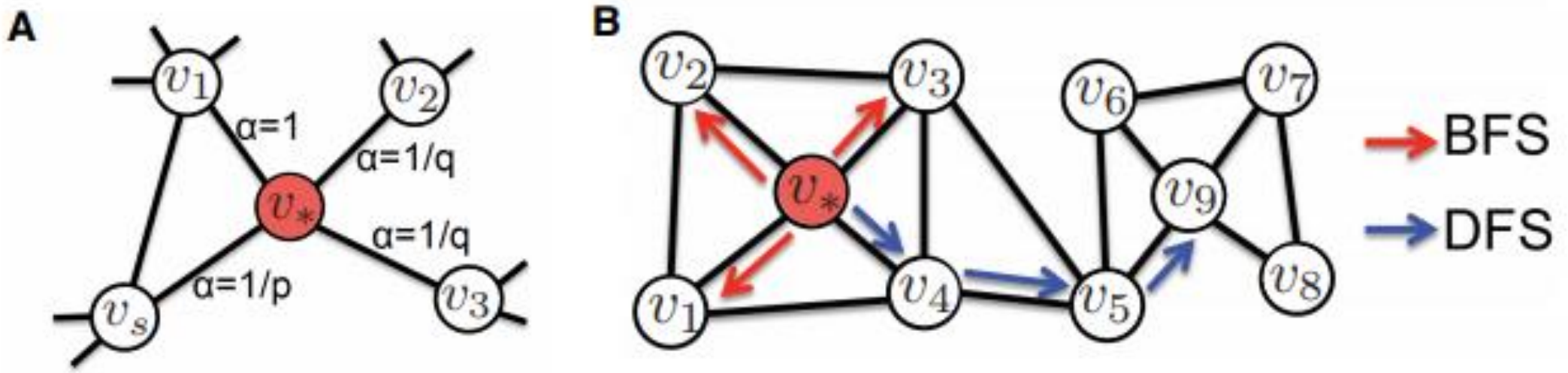


Figure 5: A, Illustration of how node2vec biases the random walk using the p and q parameters. Assuming that the walk just transitioned from v_s to v_*, the edge labels, α, are proportional to the probability of the walk taking that edge at next time-step. **B**, Difference between random-walks that are based on breadth-first search (BFS) and depth-first search (DFS). BFS-like random walks are mainly limited to exploring a node's immediate (*i.e.*, one-hop) neighborhood and are generally more effective for capturing structural roles. DFS-like walks explore further away from the node and are more effective for capturing community structures. Adapted from [28].

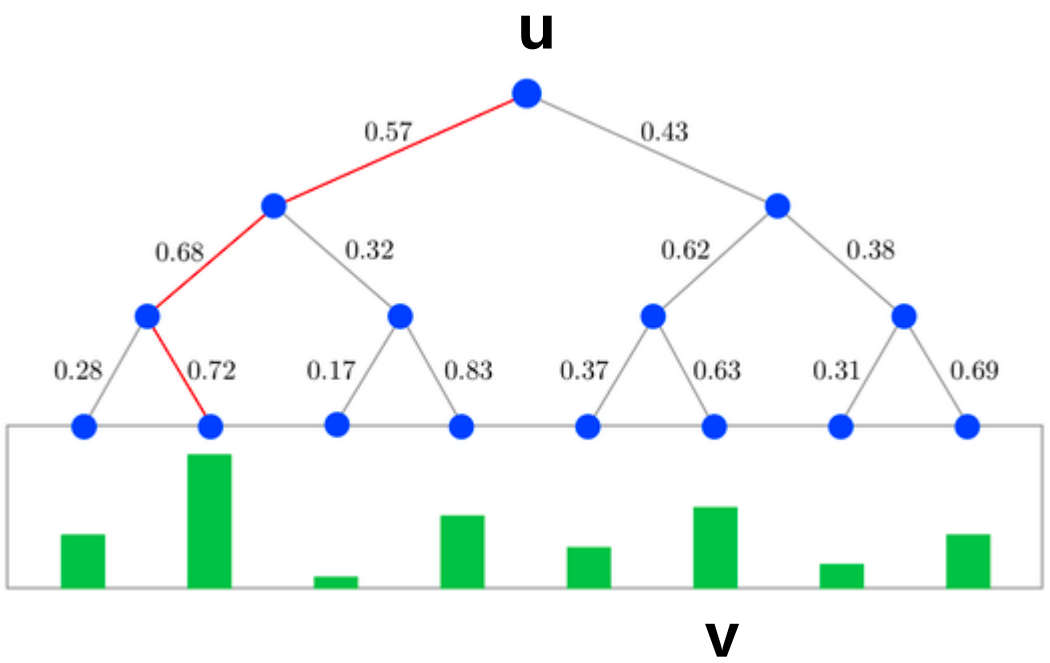
DeepWalk vs. Node2Vec: different optimization

- DW: hierarchical softmax
- N2V: negative sampling

DeepWalk: change decoder function

- DW: hierarchical softmax
- N2V: negative sampling

$$\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) \triangleq \frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{v_k \in \mathcal{V}} e^{\mathbf{z}_u^\top \mathbf{z}_k}} \approx p_{\mathcal{G},T}(v|u),$$



Node2Vec: change loss function

- DW: hierarchical softmax
- N2V: negative sampling

$$\mathcal{L} = \sum_{(u,v)\in\mathcal{D}} -\log(\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v)).$$

$$\mathcal{L} = \sum_{(u,v)\in\mathcal{D}} -\log(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)) - \gamma \mathbb{E}_{v_n\sim P_n(\mathcal{V})}[\log(-\sigma(\mathbf{z}_u^\top \mathbf{z}_{v_n}))].$$

Embedding: unifying with math

$$\mathrm{DEC}(\mathbf{z}_u, \mathbf{z}_v) = \frac{1}{1 + e^{-\mathbf{z}_u^\top \mathbf{z}_v}},$$

- See "Network Embedding as Matrix Factorization: Unifying DeepWalk, LINE, PTE, and node2vec " by Qiu (2017)

Empirics: how to implement?

- networkX or karateclub
- using keras as in node2vec

Empirics: how to implement?

- networkX or karateclub

```
from karateclub import DeepWalk

G = nx.karate_club_graph()
pos = nx.spring_layout(G)

dim = 64
dw = DeepWalk(dimensions=dim)
dw.fit(G)

embedding = dw.get_embedding()
```

Empirics: keras

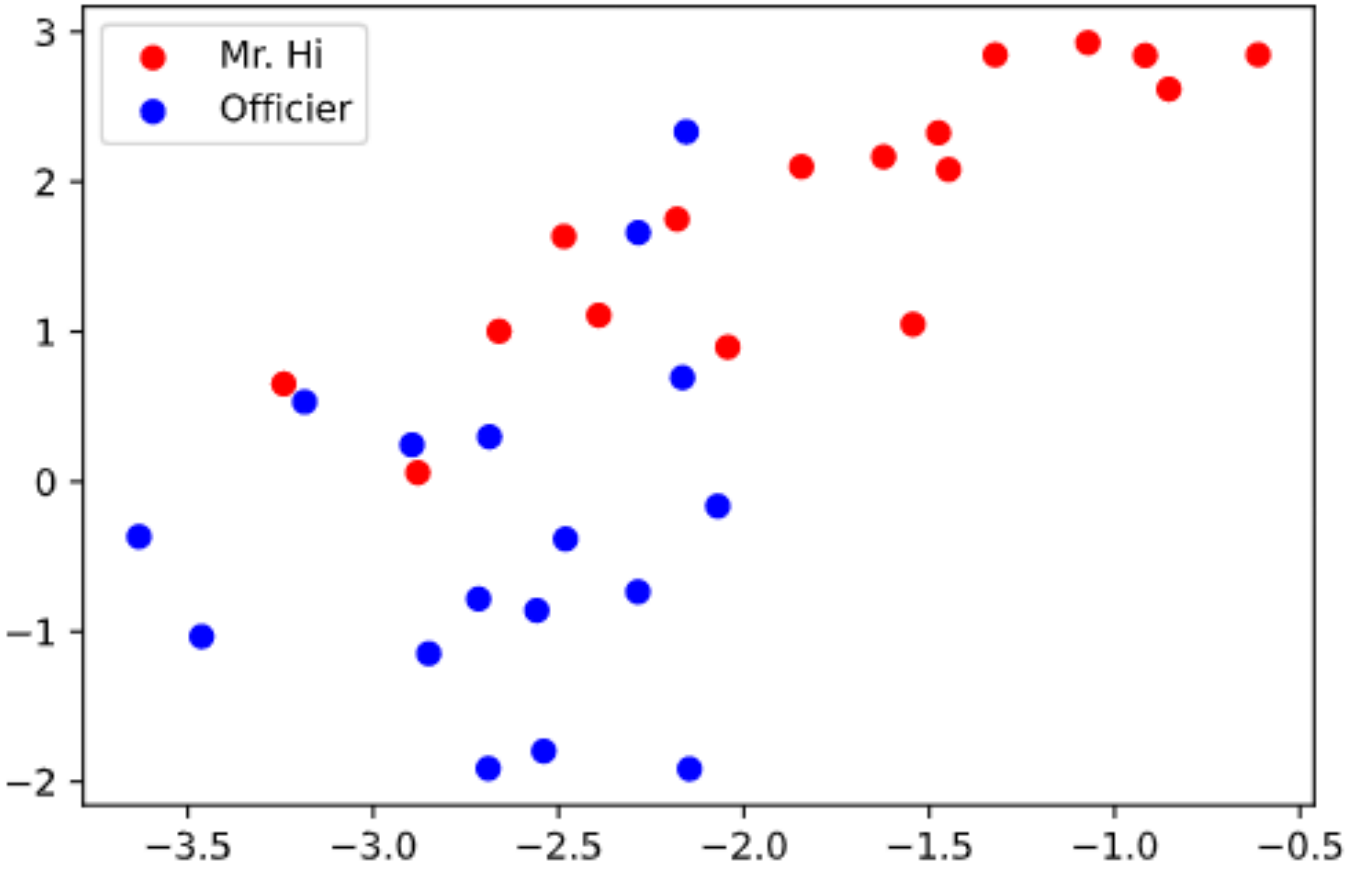
```python
def next_step(graph, previous, current, p, q):
    neighbors = list(graph.neighbors(current))
    weight = []

    for neighbor in neighbors:
        if neighbor == previous:
            weight.append(graph[current][neighbor]['weight'] / p)

        elif graph.has_edge(previous, neighbor):
            weight.append(graph[current][neighbor]['weight'])

        else:
            weight.append(graph[current][neighbor]['weight'] / q)

    probabilities = [x / np.sum(weight) for x in weight]
    next = np.random.choice(neighbors, size=1, p=probabilities)[0]

    return next
```

```python
def random_walks(graph, num_walks, num_steps, p, q):
    walks = []
    nodes = list(graph.nodes())

    for walk in range(num_walks):
        np.random.shuffle(nodes)

        for node in tqdm(nodes, desc= f'node creation {walk+1} of {num_walks}'):
            walk = [node]

            while len(walk) < num_steps:
                current = walk[-1]
                previous = walk[-2] if len(walk) >1 else None
                next = next_step(graph, previous, current, p, q)
                walk.append(next)
            walk = [vocabulary_lookup[token] for token in walk]

            walks.append(walk)

    return walks
```

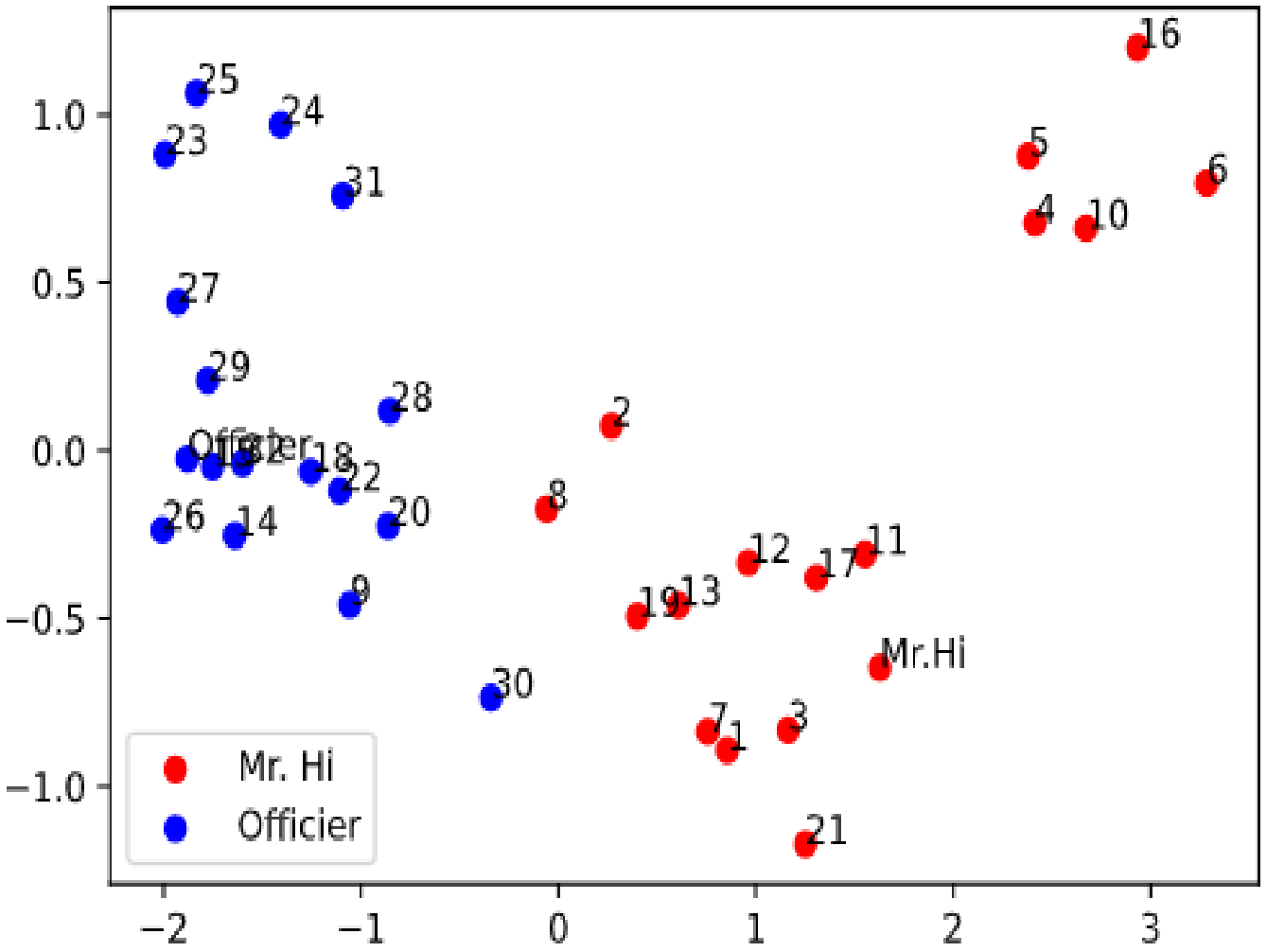
Empirics: how to implement?

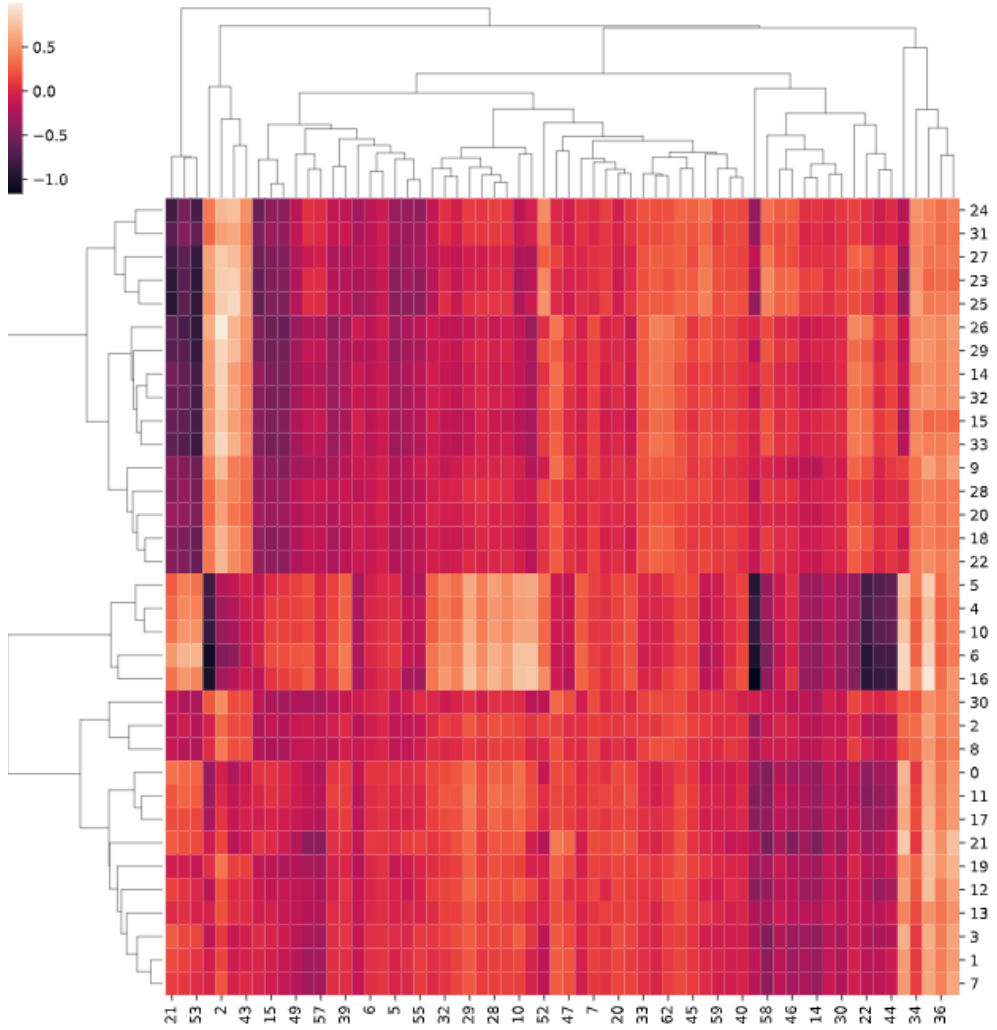
- networkX or karateclub
- using keras as in node2vec
  - see graph_node2vec.pptx

2D-embedding: karate club with deepwalk(dimension=64) and pca



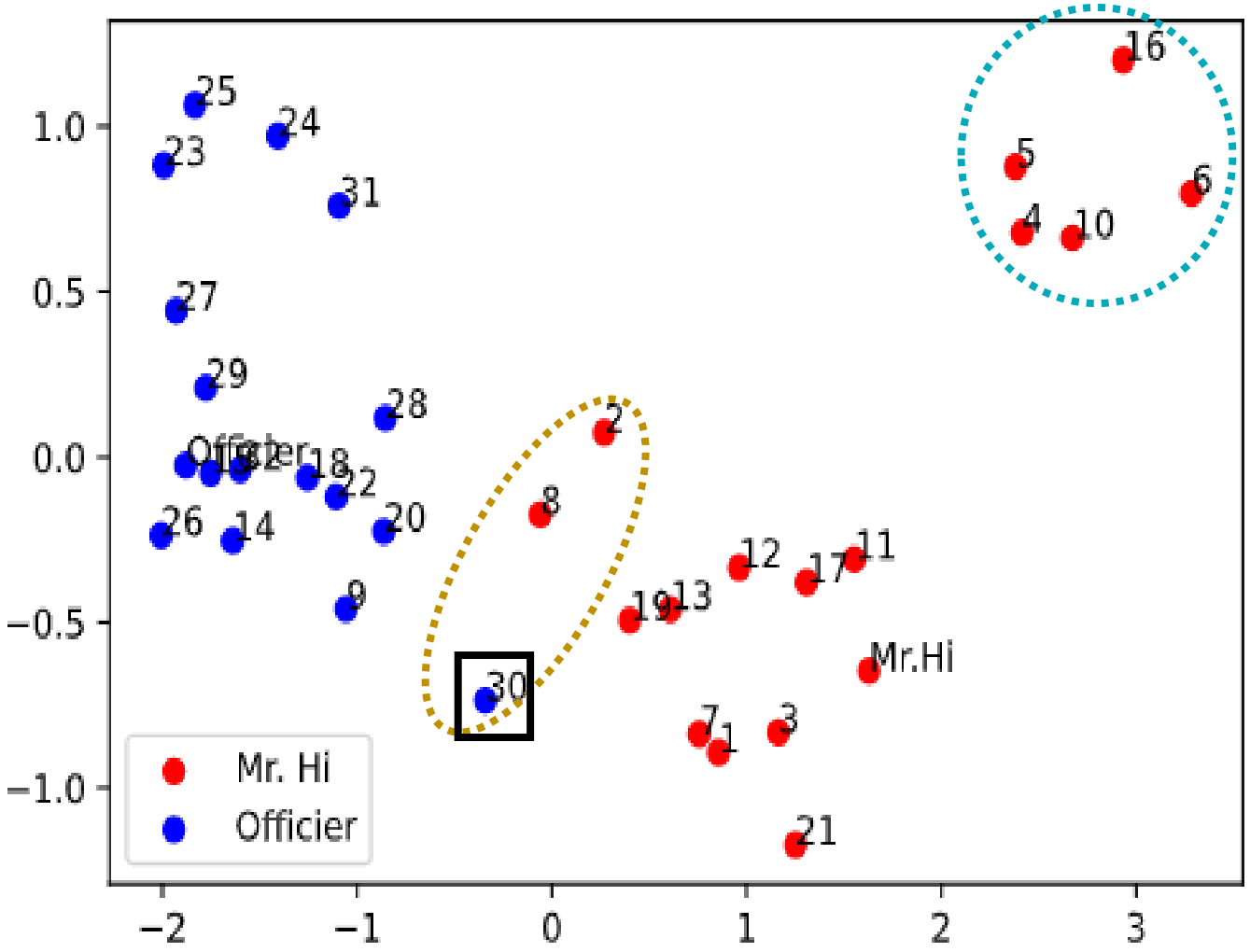
Embedding into 2-D (PCA)

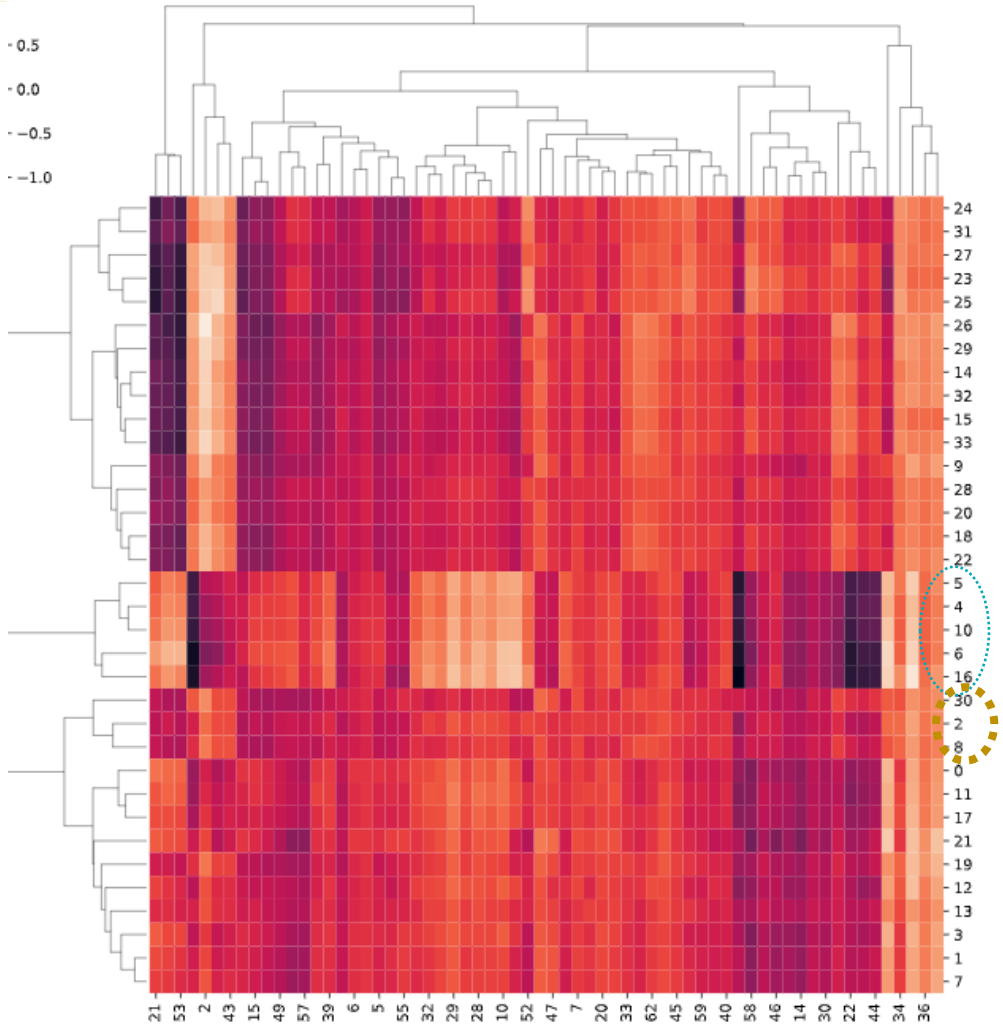




eigenvector heatmap

Embedding into 2-D (PCA)





eigenvector heatmap

Limitations of shallow embedding

- no parameter sharing in node
  - not leverage node features in encoder
- not inductive
  - **transductive**: only generate embedding during training stage. Hence, not possible to inductive applications, such as, unseen-node classification

Deep Embeddings

- Why deeplearning?
- Models
  - CNN
  - Group-Equivalent CNNs
  - Graph NN
  - Transformers
  - Message Passing
  - RNN
  - LSTM

Limitations of shallow embedding

- no parameter sharing in node
  - not leverage node features in encoder
- not inductive
  - **transductive**: only generate embedding during training stage. Hence, not possible to inductive applications, such as, unseen-node classification

References

- Blogs
  - Thomas Kipf
  - Michael Bronstein OR Geometric Deeplearning
  - Petar V. (Deepmind)
- Books / papers
  - Bronstein et. al (manuscript)
  - W. Hamilton book
  - Deep Learning on Graphs (Ma and Tang, 2020, manuscript)
  - A comprehensive survey on graph neural networks (Wu et. al, 2020)
- Library
  - Graph Nets
  - Spektral
  - DGL
  - PyTorch Geometric

References

- Lecture
  - Xavier Bresson guest lecture at NYU
  - Petar: Introduction to GNN
  - Bronsteim, 2020 or googling with his name
  - Stanford CS224W (Leskovec)

CNN Review

Input layer

1	2	3	4	5	6	7
8	9	10 0.1	11 0.2	12 0.3	13	14
15	16	17 0.4	18 0.5	19 0.6	20	21
22	23	24 0.7	25 0.8	26 0.9	27	28
29	30	31	32	33	34	35
36	37	38	39	40	41	42
43	44	45	46	47	48	49

Convolution

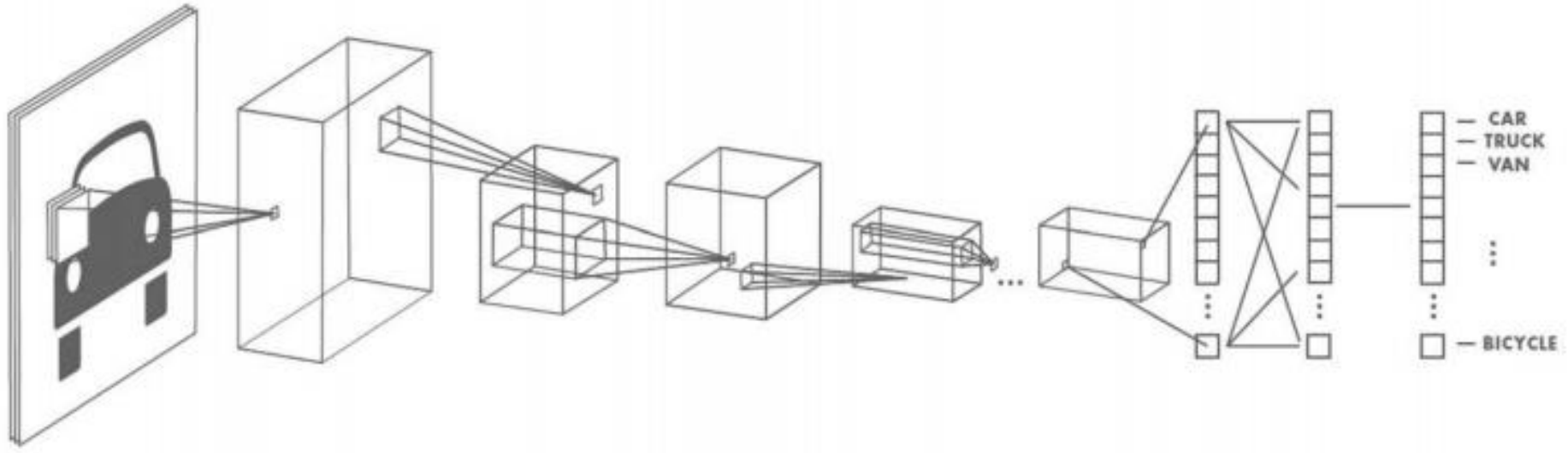
0.1	0.2	0.3
0.4	0.5	0.6
0.7	0.8	0.9

Input layer: convolution

1	2	3	4	5	6	7
8	9	10 0.1	11 0.2	12 0.3	13	14
15	16	17 0.4	18 0.5	19 0.6	20	21
22	23	24 0.7	25 0.8	26 0.9	27	28
29	30	31	32	33	34	35
36	37	38	39	40	41	42
43	44	45	46	47	48	49

CNN data:

- **local:**
  - **local receptive field assumption**
  - **local invariance**
- **stationarity (global)**
  - **similar patterns and shared**
  - **translation invariant**
- **hierarchical**
  - **sub-level features combined to for upper-level features**

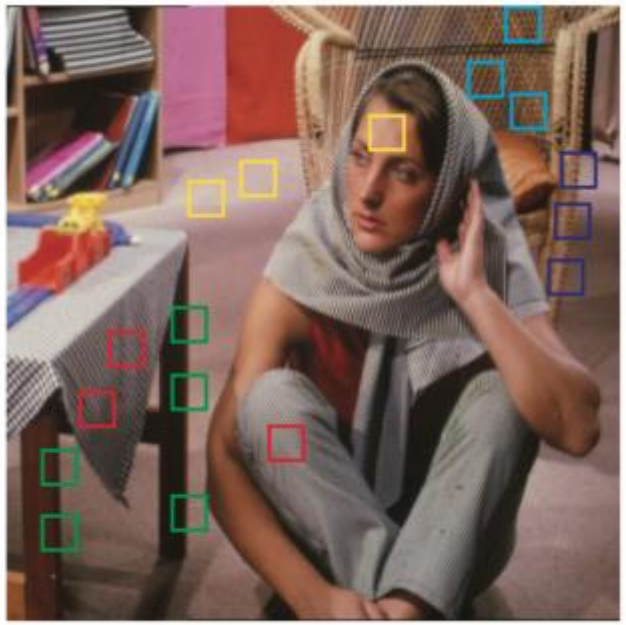


LeCun et al. 1989

stationarity: global vs. local

- **stationarity (global)**
  - **similar patterns and shared**
  - **translation invariant**

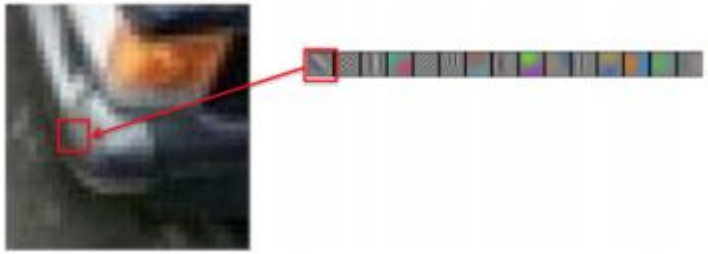
Anywhere
-rotation
-mirroring
-translation



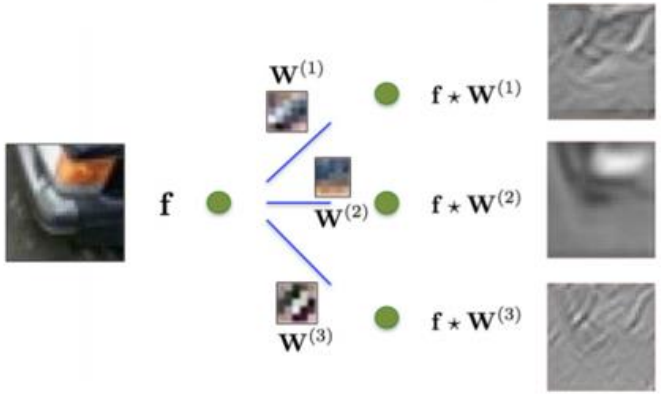
- **local:**
  - **local receptive field assumption**
  - **local invariance**

CNN implementation

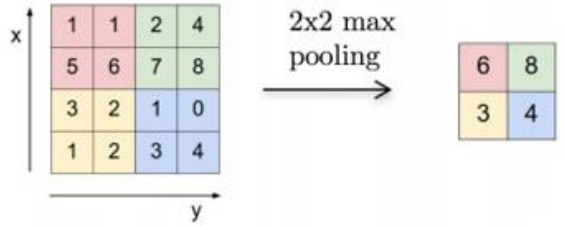
- **locality**



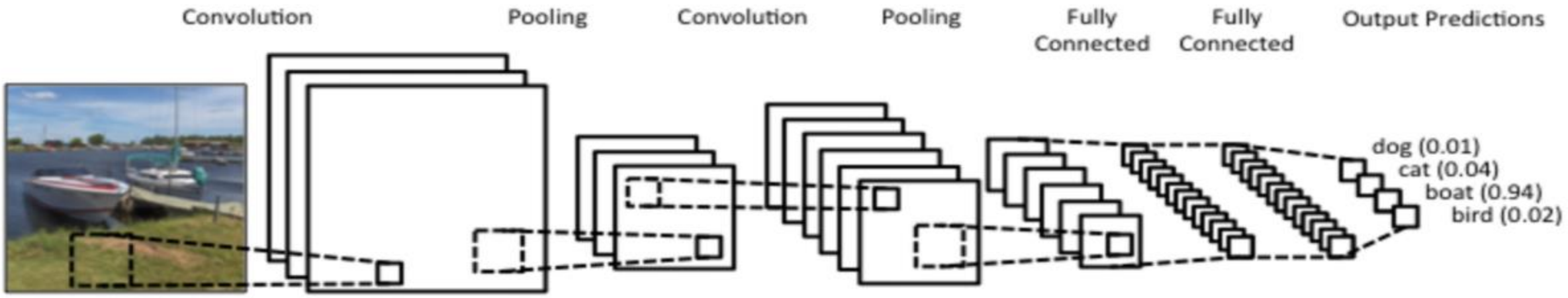
- **stationarity**



- **multi-scale**
  (down sampling)



ConvNet



- filters localized in space (locality)
- convolution filters (global stationarity)
- Multiple layers
- independent of input image size n
- filtering is done in spatial domain

Graph data

CNN data

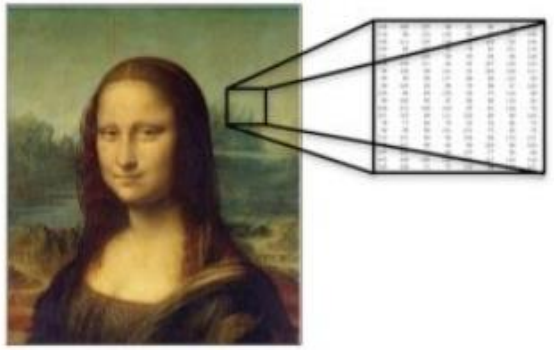
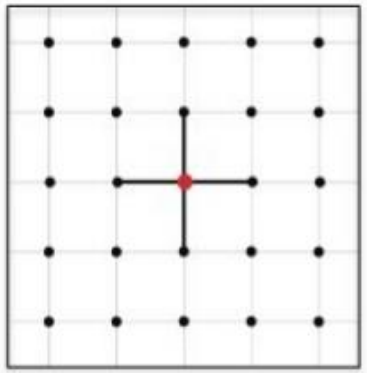
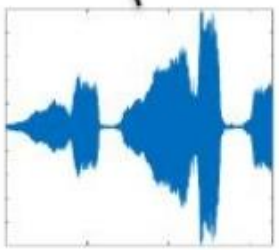


Image.



2D grids

Sentence.



sound:

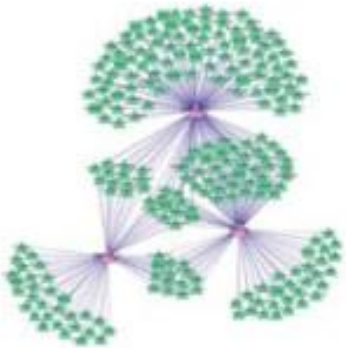
1D grid

Euclidean Domains

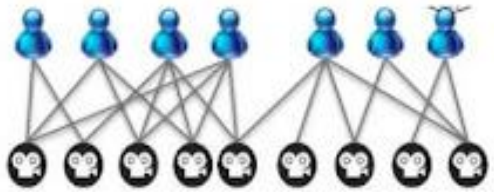
CNN on graph?

Non-Euclidean data a.k.a. Graphs/Networks

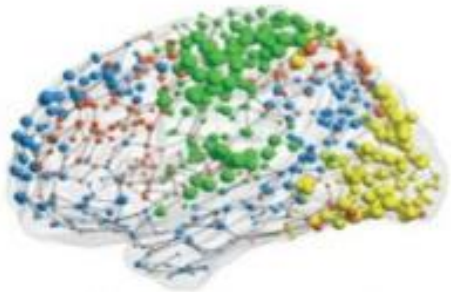
Social networks



Regulatory networks

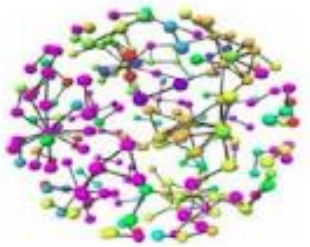


Retail networks



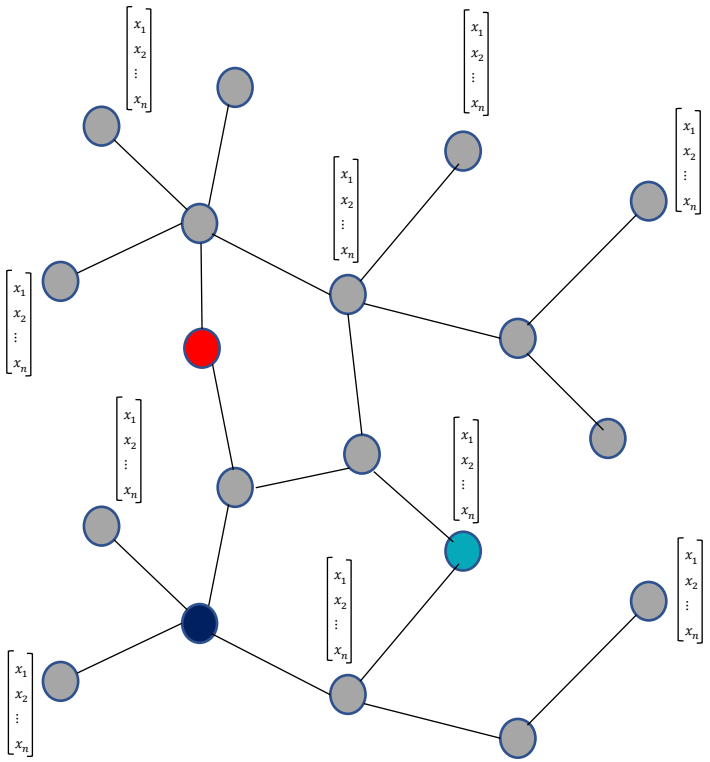
Functional networks

3D shapes



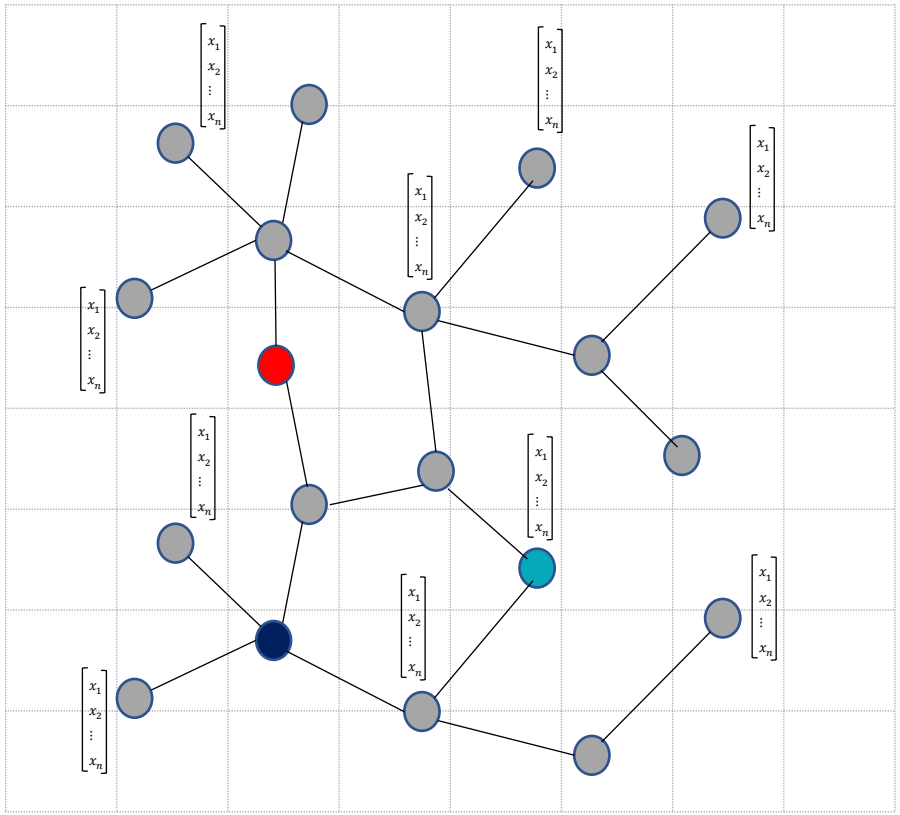
Biological networks

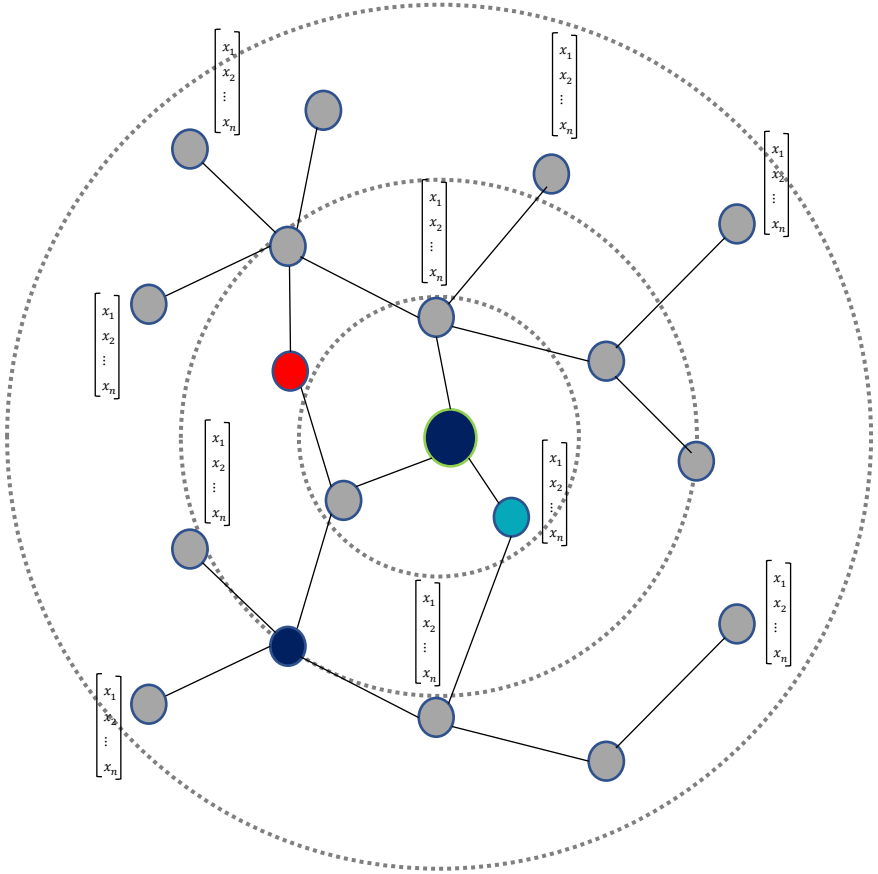
Graph data



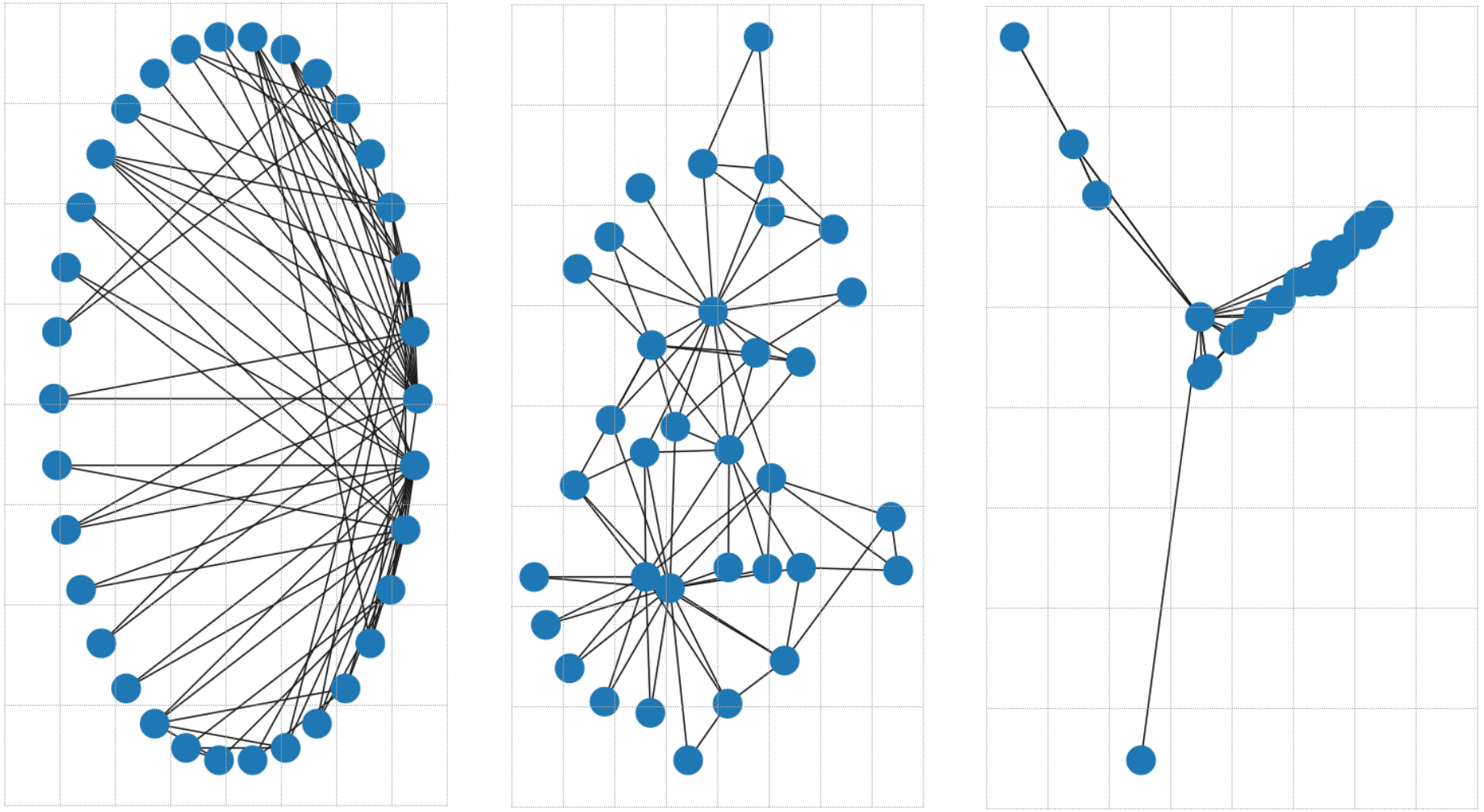
- Graph: (vertex, edge, adj. mat)= (V, E, A)
- Graph features
  - node features: $\mathbf{h_i}, \mathbf{h_j}$
  - edge feature: $\mathbf{e_{ij}}$
  - graph features

How to convolve?





same graph different layout

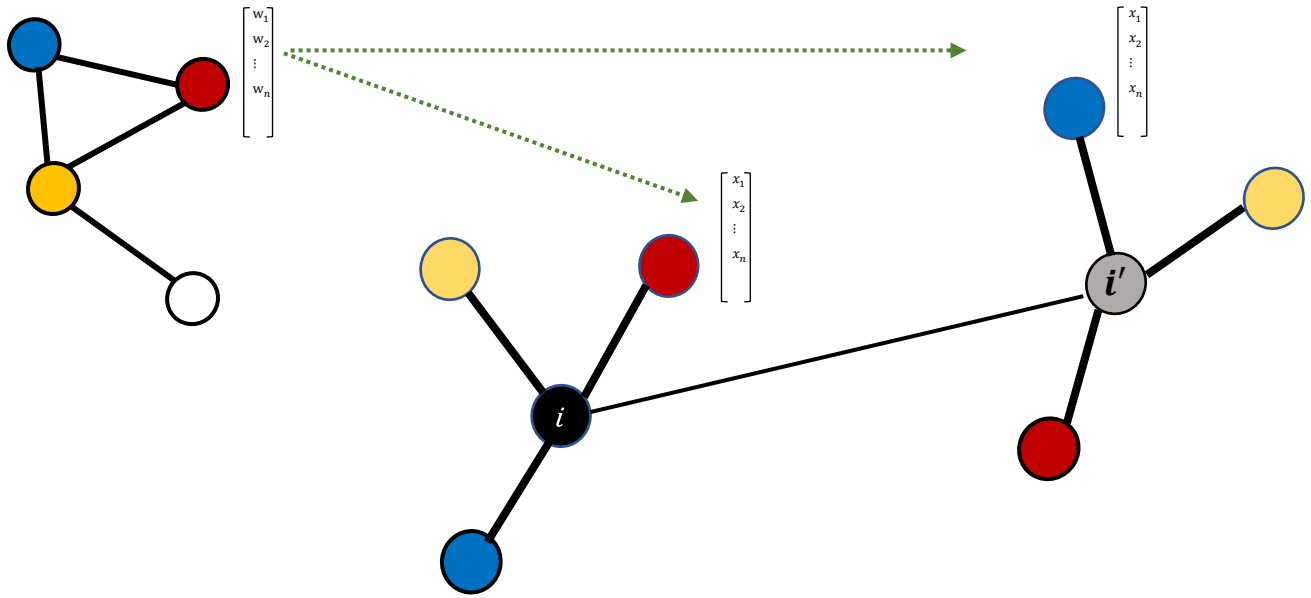


CNN is not applicable to graph

- There is no node ordering
- neighborhood sizes may be different

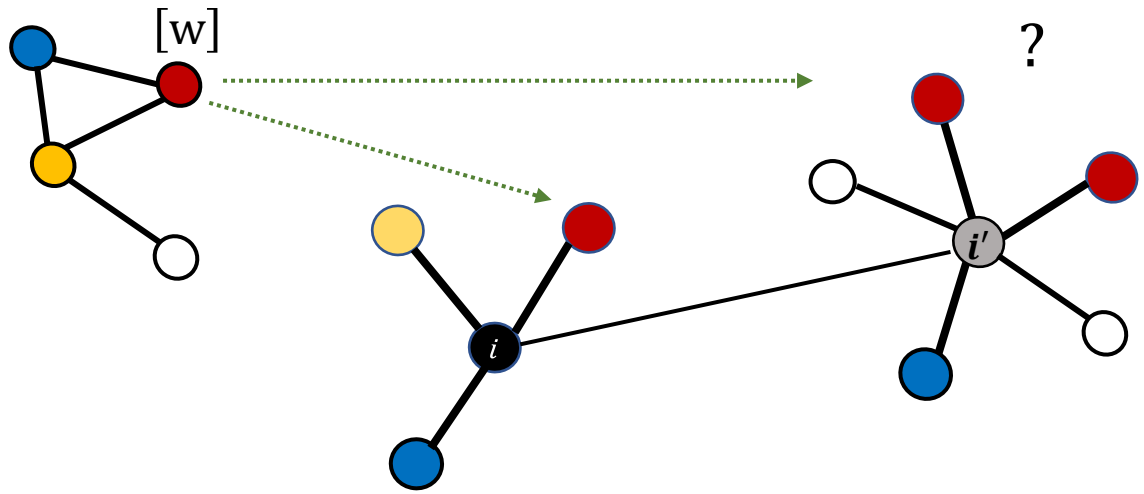
CNN is not applicable to graph

- There is no node ordering
- neighborhood sizes may be different



CNN is not applicable to graph

- There is no node ordering
- neighborhood sizes may be different



GCN: spectral

- Spectral GCN
  - vanilla
  - LapGCN
  - SplineGCN
  - Chebyshev Polynomials
  - CayleyNets
  - ....

GCN:math

Graph (normalized) Laplacian

$$\underset{n\times n}{\Delta} = I - D^{-1/2} A D^{-1/2}$$

where

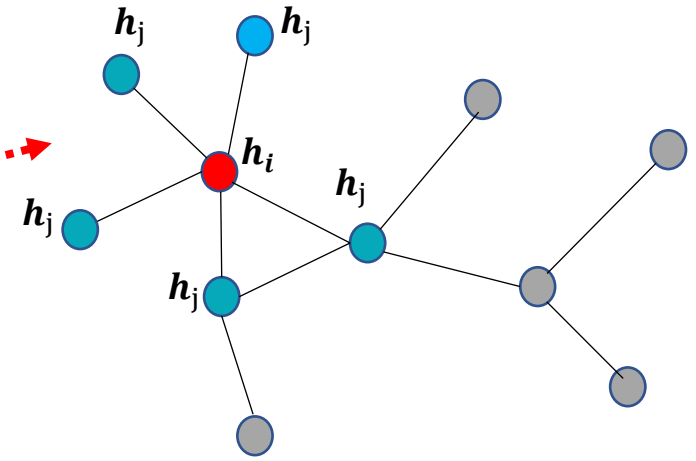
$$A = \text{adjacency matrix},$$

$$D = \text{degree matrix} = \text{diag}\left(\sum_{j\neq i} A_{ij}\right)$$

- Δ is the measurement of smoothness of graph; the difference b/w the local value h_i and its neighborhood value of $h_j's$

$$(\Delta h)_i = h_i - \sum_{j\in\mathcal{N}_i} \frac{1}{\sqrt{d_i d_j}} A_{ij} h_j \qquad (1)$$

- Equation (1) will be used on the graph nodes; applying Laplacian on a node h_i is the value of h_i minus the mean value over its neighboring nodes $h_j's$



Laplacian smoothing (L)

Fourier functions

- eigen decomposition of Laplacian

$$\Delta = \overbrace{\Phi^T}^{\substack{\text{Graph} \\ \text{Fourier Basis}}} \underbrace{\Lambda}_{\substack{\text{Graph} \\ \text{frequency}}} \Phi$$

where

$$\Delta = (\text{normalized}) \text{ Laplacian matrix},$$
$$\Phi = \left[\phi_1 | \cdots | \phi_n\right]_{(n \times n)},$$
$$\phi_k = (n \times 1) \text{ eigenvector OR Fourier functions}$$
$$\underset{n \times n}{\Lambda} = diag(\lambda_1, \cdots, \lambda_n), \quad 0 \leq \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n = \lambda_{\max}$$

coro: $\Delta \phi_i = \lambda_k \phi_i, \quad \forall i = 1, 2, \cdots, n$

$$\omega * h = \hat{\omega}(\Delta) \cdot h = \Phi \hat{\omega}(\Lambda) \Phi^T$$

convolution theorem

> space convolution = frequency multiplication

$$\mathcal{F}(\omega * h) = \mathcal{F}(\omega) \odot \mathcal{F}(h) \quad (\odot = \text{element-wise product})$$
$$\Longrightarrow \omega * h = \mathcal{F}^{-1}\left(\mathcal{F}(\omega) \odot \mathcal{F}(h)\right)$$

- how to being defined graph convolution in the spectral domain?
  - graph convolutions can be expressed as polynomials of the Laplacian (normalized)

Theorem: $w * h = \hat{w}(\Delta)h$

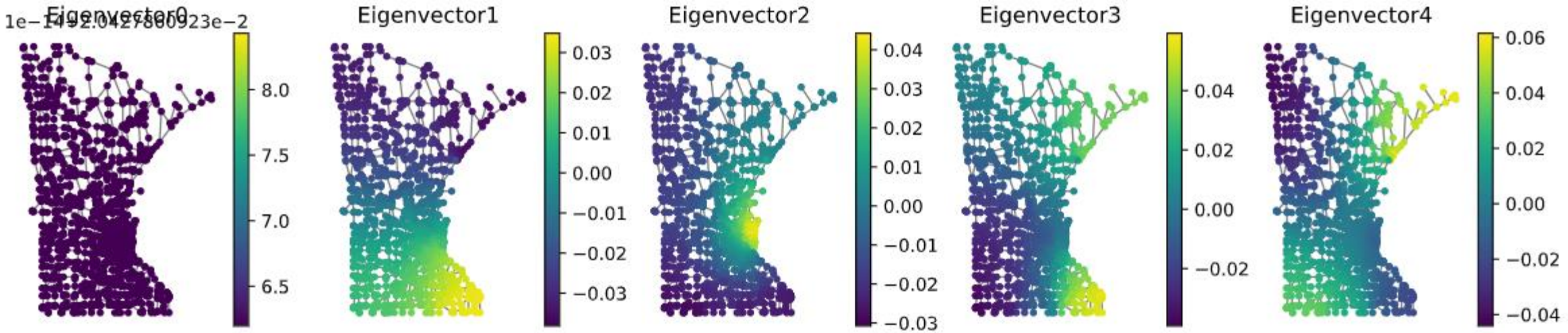
$$\begin{aligned}
\underset{n\times 1}{\omega} * h &= \mathcal{F}^{-1}\Big(\mathcal{F}(\omega) \odot \mathcal{F}(h)\Big) \\
&= \underset{n\times n}{\Phi}\Big(\Phi^T \underset{n\times 1}{\omega} \odot \Phi^T h\Big) \\
&\equiv \Phi\Big(\underset{n\times 1}{\hat{\omega}} \odot \Phi^T \underset{n\times 1}{h}\Big) \\
&= \Big(\Phi\ \mathrm{diag}(\hat{\omega_1}, \cdots, \hat{\omega_n})\Phi^T h\Big) (\text{non-parametric representation of filter})
\end{aligned}$$

$$\omega * h = \Phi \begin{bmatrix} \hat{\omega}_1 & & & \\ & \hat{\omega}_2 & & \\ & & \ddots & \\ & & & \hat{\omega}_n \end{bmatrix} \Phi^T h$$

FACT:

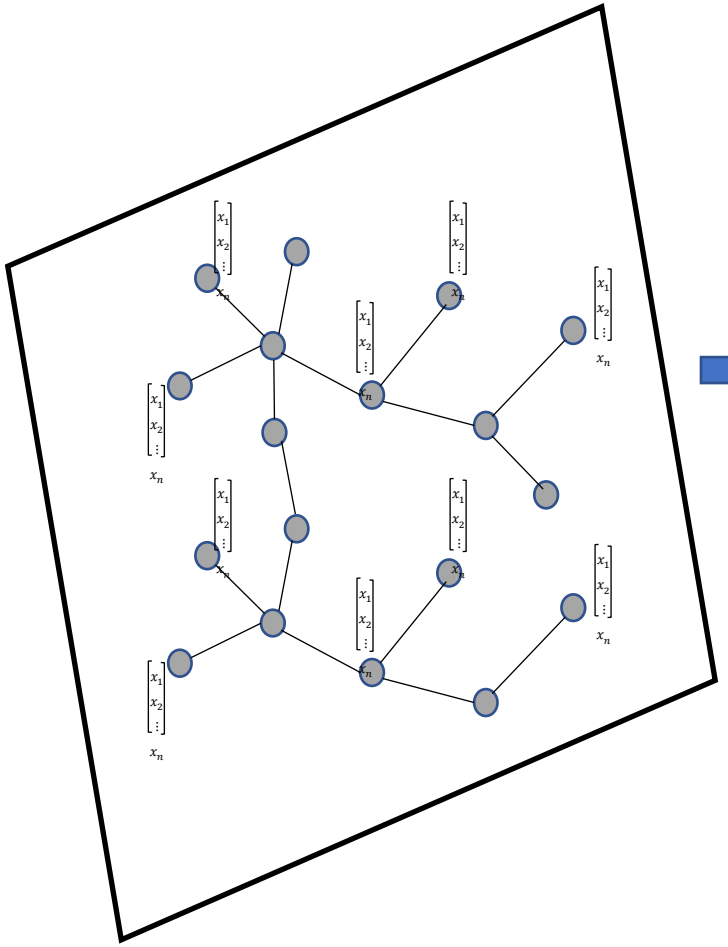
$$\omega * h = \hat{\omega}(\Delta) \cdot h = \Phi \hat{\omega}(\Lambda)\Phi^T$$

Eigenvectors of Laplacian (Minnesota Road)



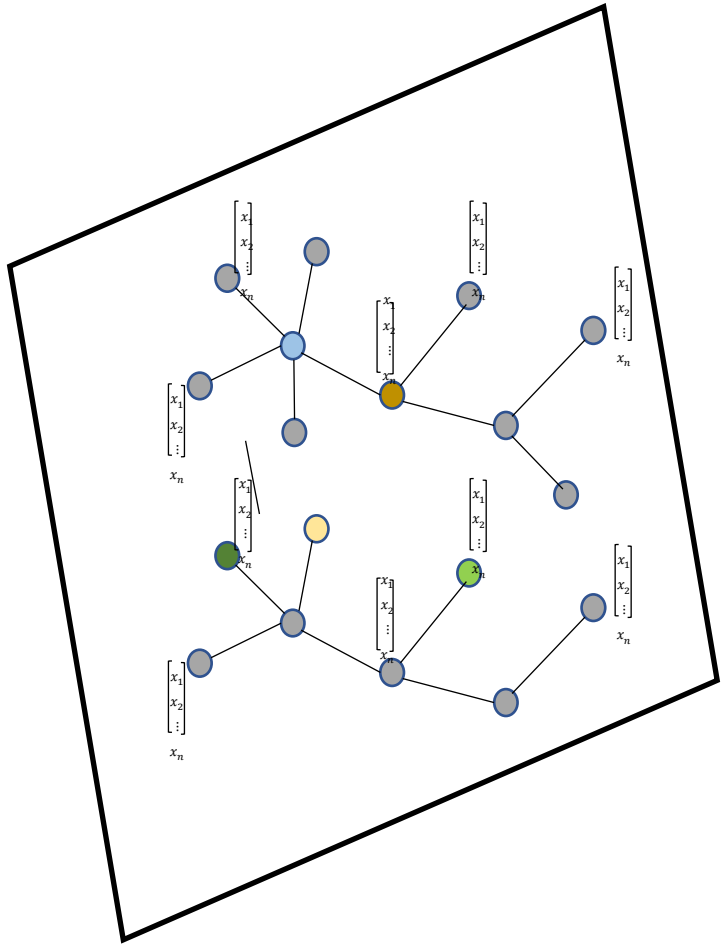
Fiedler vector

Graph Filtering



Graph Filtering

Graph structure remained,
Features are filteredg



Graph Fourier Transform

The process of filtering (spectral here)

[step1]: signal h, Φ(with Laplacian)

[step2]: project signal onto eigenspace

$$h = \Phi^T h = \sum_k \phi_k h$$

[step3]: filter with $\hat{\omega}(\Lambda)$

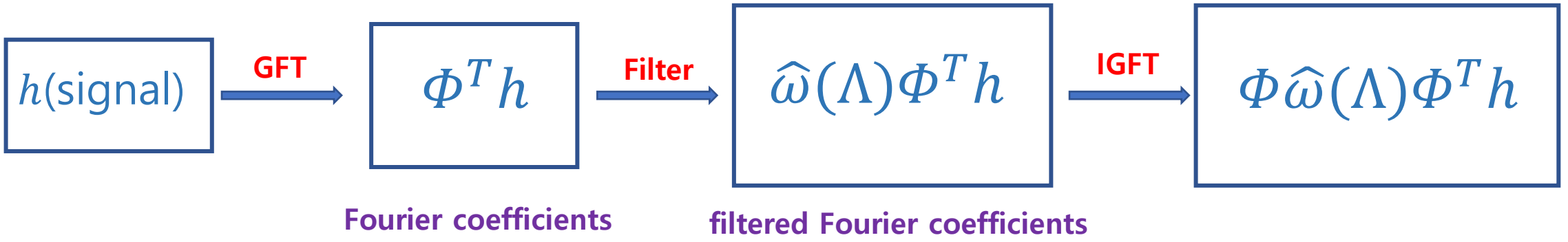
[step4]: get filtered coefficients

$$\hat{\omega}(\Lambda)\Phi^T h$$

[step5]: Reconstruct graph signal

$$\Phi\hat{\omega}(\Lambda)\Phi^T h$$

Meaning of Graph Fourier Transform



Spectral GCN

Vanilla spectral GCN

- Graph spectral convolution layer

$$
\begin{aligned}
\underset{n\times d}{h^{l+1}} &= \eta\left(\omega^l * h^l\right) \\
&= \eta\left(\hat{\omega}^l(L)h^l\right) \\
&= \eta\left(\underset{n\times n}{\Phi}\ \underset{n\times n}{\hat{\omega}^l(\Lambda)}\ \underset{n\times d}{\Phi h^l}\right)
\end{aligned}
$$

spatial filter

spectral filter

$$
\hat{\omega}(\Lambda) = \begin{bmatrix} \hat{\omega}(\lambda_1) & & & \\ & \hat{\omega}(\lambda_2) & & \\ & & \ddots & \\ & & & \hat{\omega}(\lambda_n) \end{bmatrix}
$$

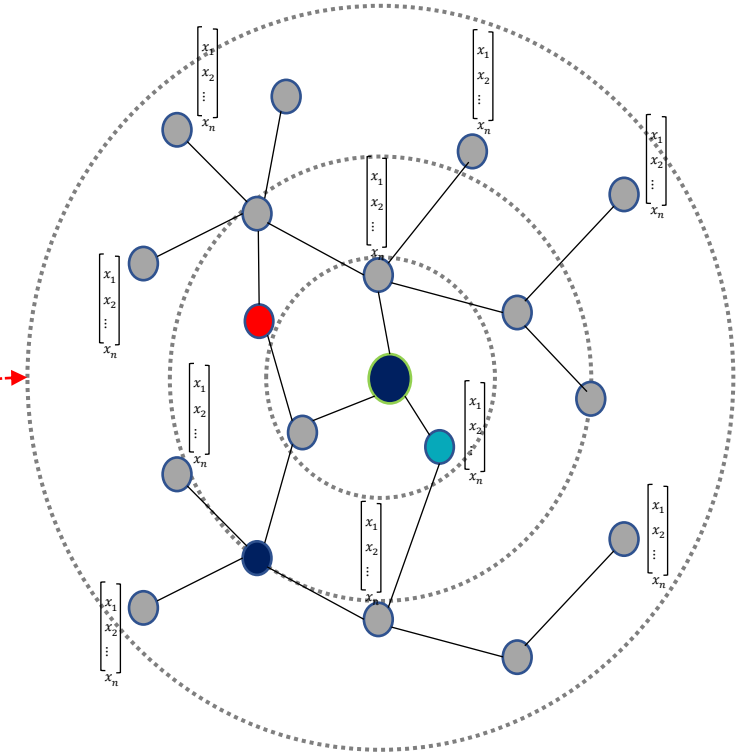
LapGCN

- localized spectral filterings

$$\omega * h = \hat{\omega}(\Delta)h$$

$$\hat{\omega}(\Delta) = \sum_{k=0}^{K-1} \omega_k \Delta^k$$

- filters are exactly localized k-hop supports
- ω_k are learned by backpropagation



Limitations of spectral nn

- Learned filters may not be generalized across graphs
  - filters are basis- dependent
- Computationally expensive ($\mathcal{O}(n^2)$)
- Definition of the directed Laplacian is unclear
- Spectral techniques work with fixed size graps

GNN:

- Overview
  - information gathering
  - convolve
  - network motif
- GNN
  - message passing update
  - GraphSage
  - GCN
  - GAT

Overview

Message passing update

$h_u^{(l)}$:hidden embeddings for node$u \in \mathcal{V}$

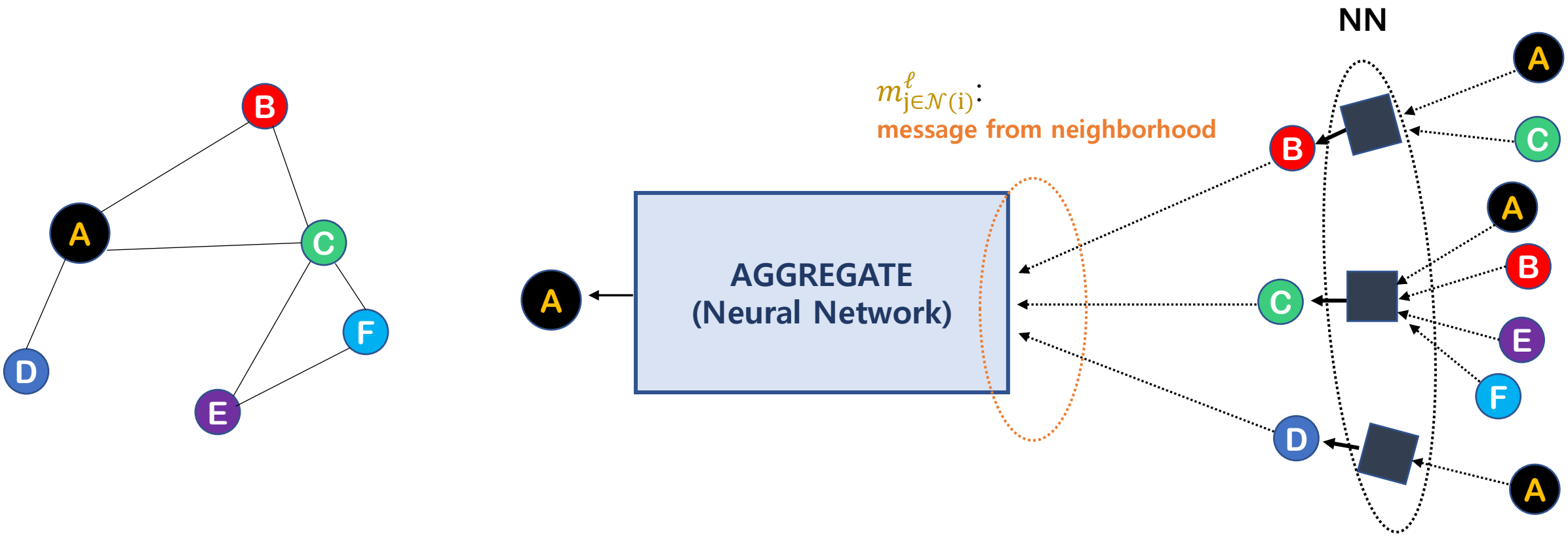
$$h_i^{l+1} = \text{UPDATE}\left(h_i^l, \text{AGGREGATE}_{j \in \mathcal{N}(i)} h_j^l\right)$$

$$\equiv \text{UPDATE}\left(h_i^l, m_{j \in \mathcal{N}(i)}^l\right)$$

where AGGREGATE and UPDATE are differentiable functions (neural network),

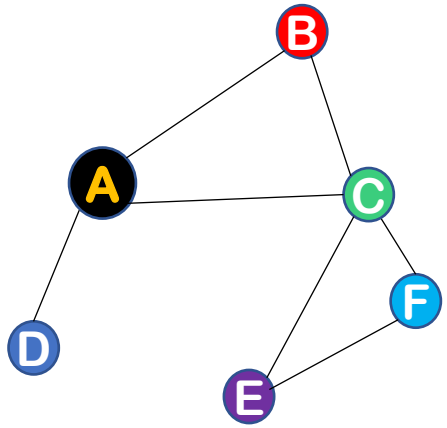
$m_{j \in \mathcal{N}(i)}^l$ is ***message*** on neighborhood j

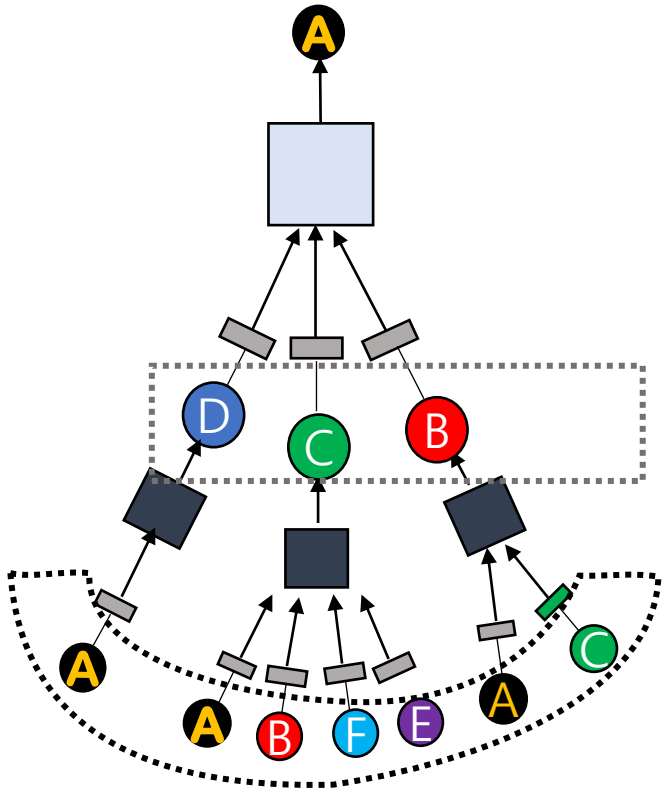
Intution: *gather information from neighborhood*



how to gather information?

- ℓ =1: one-hop neighborhood information
- after ℓ iterations, every node embedding contains information about ℓ-hop neighborhood

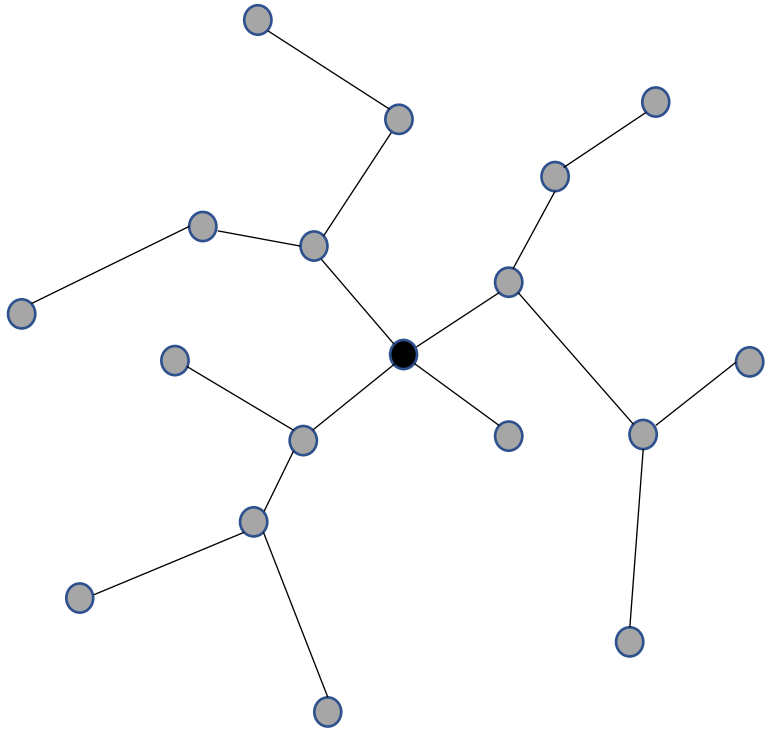


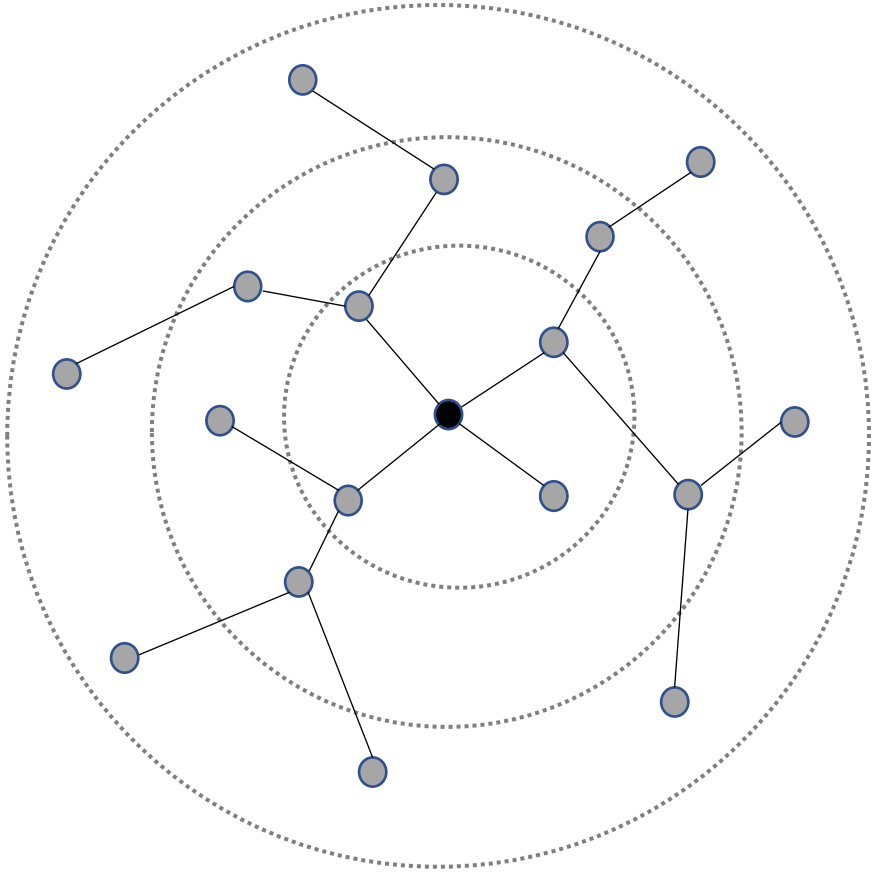


$\ell = 1$

$\ell = 2$

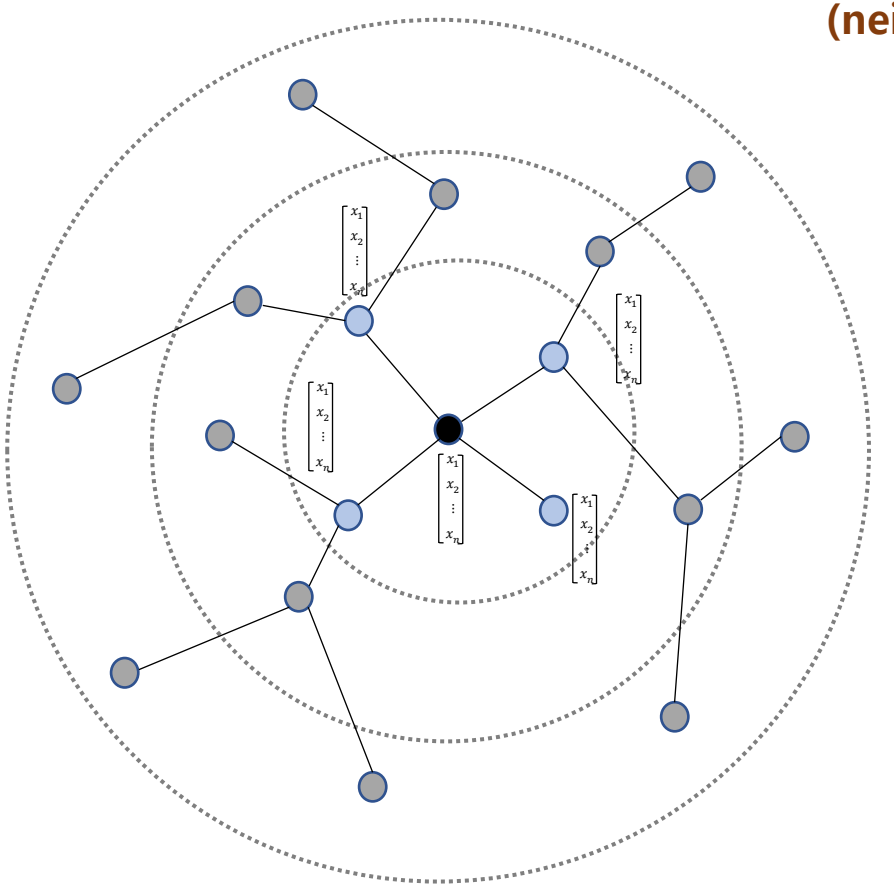
how to convolve?

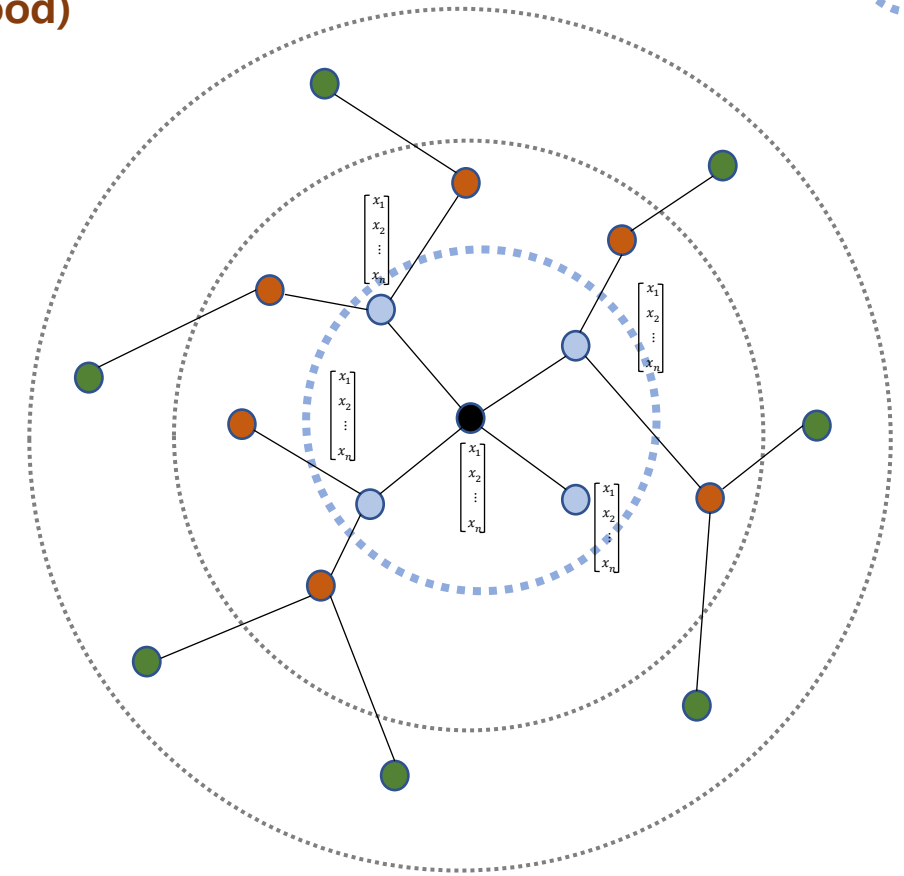




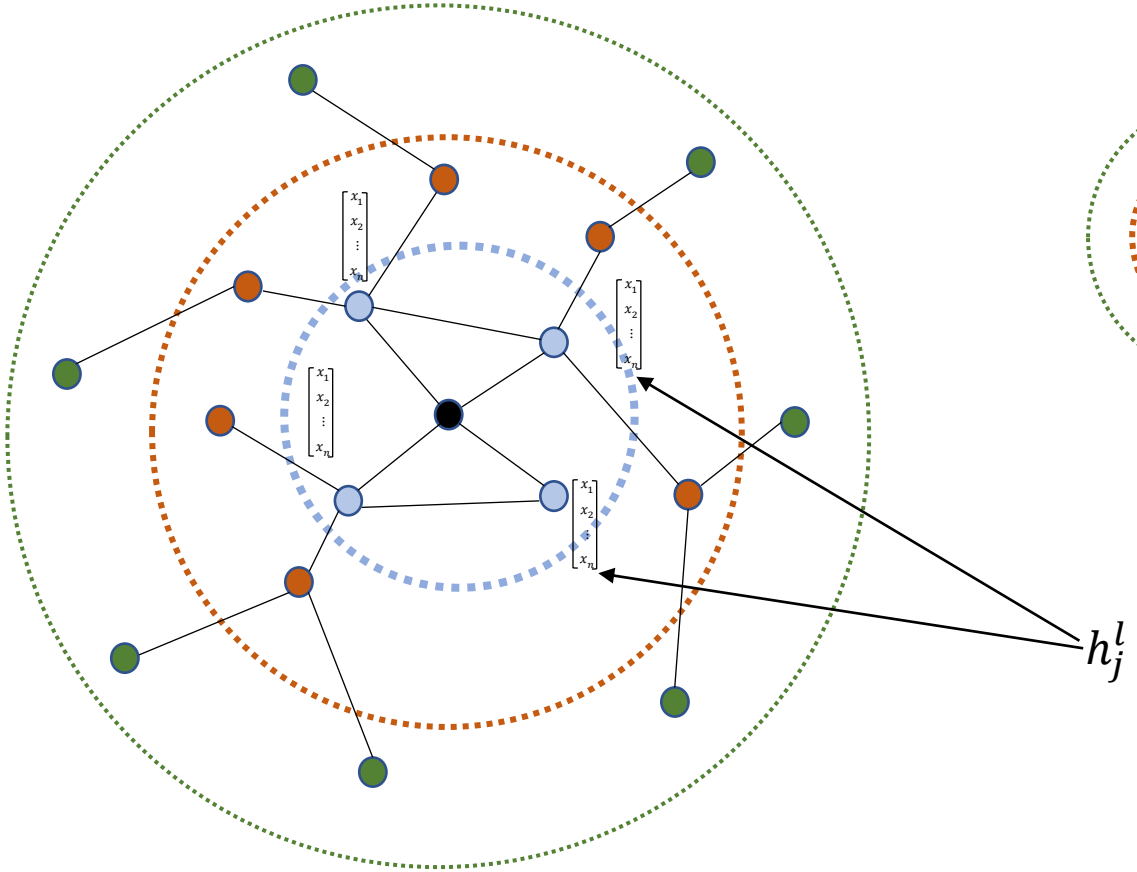
how to convolve?

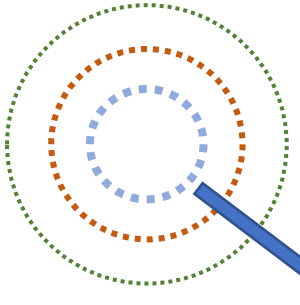
define convolution with edge connections (neighborhood)





Graph convolution



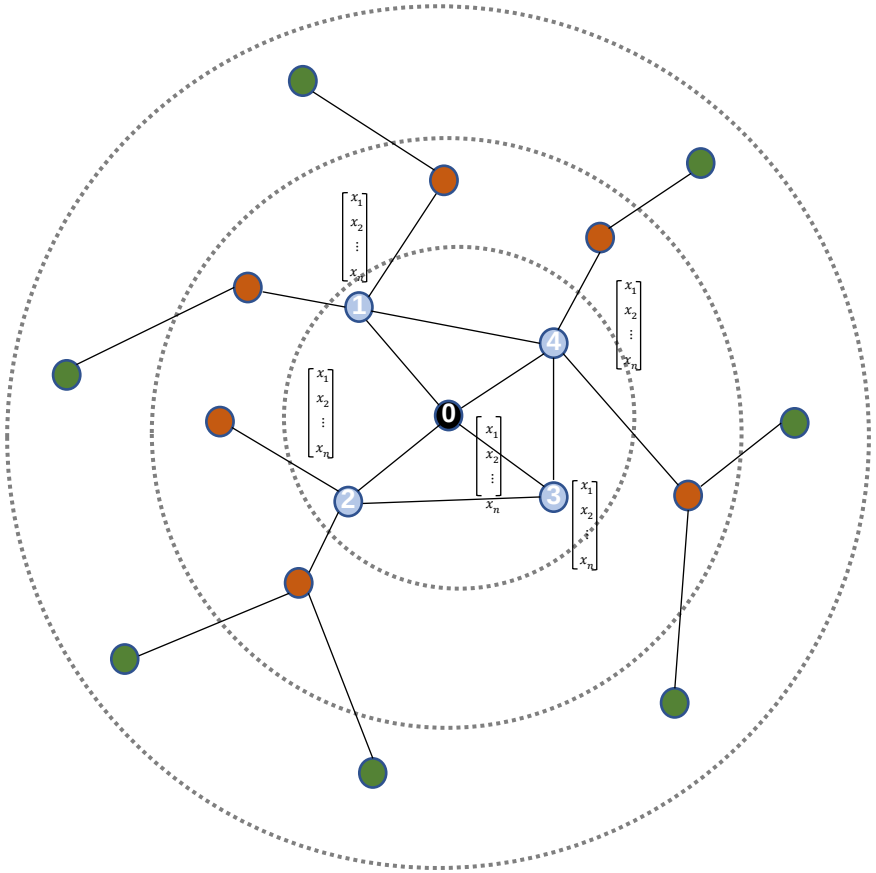


CONVOLUTION

- signal
- filter

$$h_i^{l+1} = \eta\left(\sum_{j\in\mathcal{N}(i)} A_{ij} W^l h_j^l\right)$$

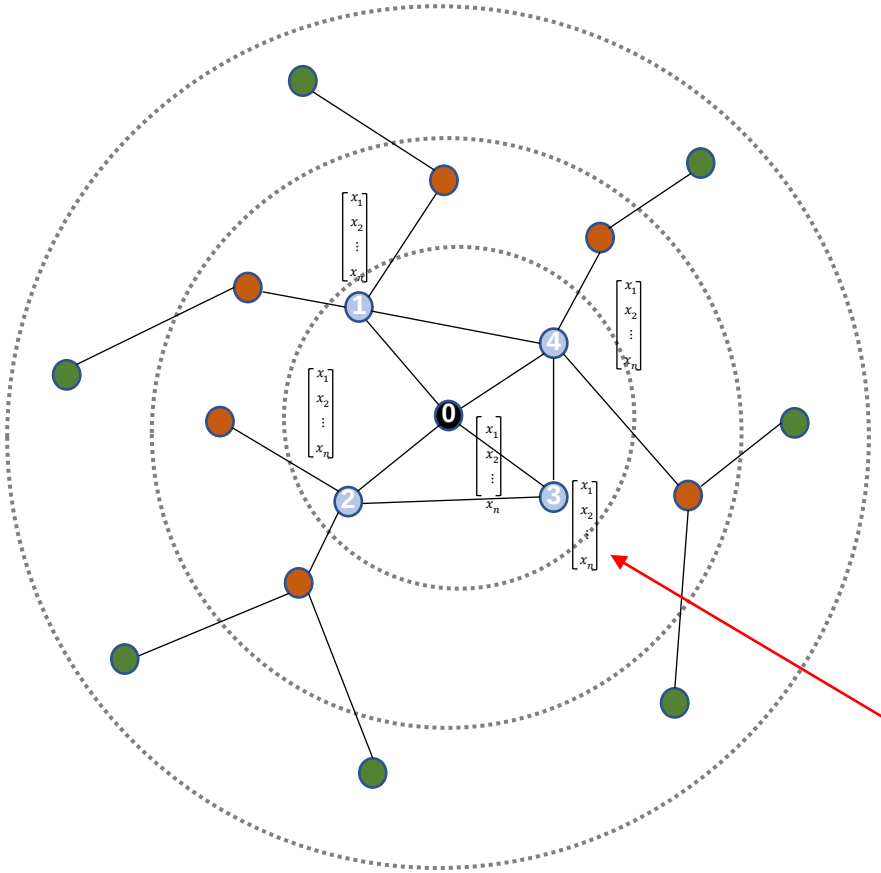
$$h_i^{l+1} = \eta\left(\sum_{j\in\mathcal{N}(i)} A_{ij} W^l h_j^l\right)$$



$$A = [A_{ij}] =$$

	0	1	2	3	4
0	0	1	1	1	1
1	1	0	0	0	1
2	1	0	0	1	0
3	1	0	1	0	1
4	1	1	0	1	0

$$h_i^{l+1} = \eta\left(\sum_{j\in\mathcal{N}(i)} A_{ij} W^l h_j^l\right)$$



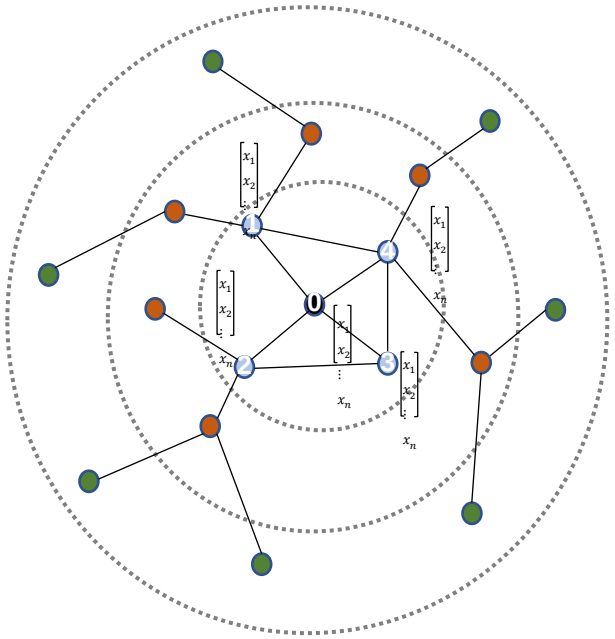
$A = [A_{ij}] =$

	0	1	2	3	4
0	0	1	1	1	1
1	1	0	0	0	1
2	1	0	0	1	0
3	1	0	1	0	1
4	1	1	0	1	0

$h^l =$

```
[[0. 0. 0. 0. 1.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [1. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0.]]
```

$$h_i^{l+1} = \eta\left(\sum_{j\in\mathcal{N}(i)} A_{ij} W^l h_j^l\right)$$



$A = [A_{ij}] =$

	0	1	2	3	4
0	0	1	1	1	1
1	1	0	0	0	1
2	1	0	0	1	0
3	1	0	1	0	1
4	1	1	0	1	0

$h^l =$

```
[[0. 0. 0. 0. 1.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [1. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0.]]
```

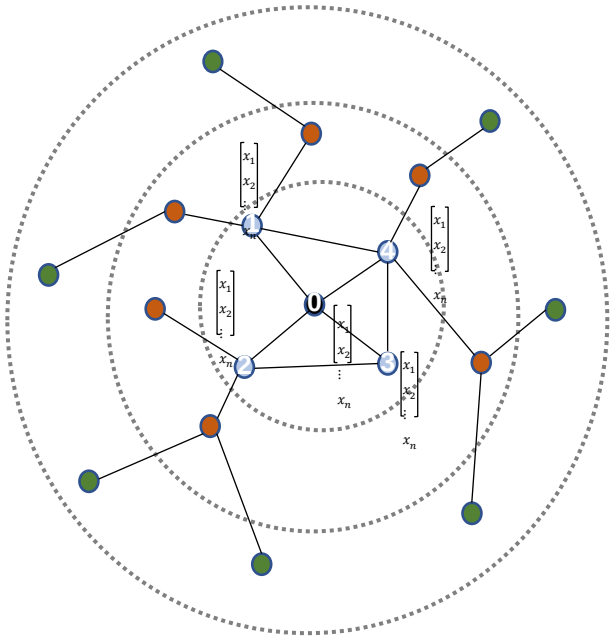
optimal weight
thru backpropagation

$W^l = [W_{ij}] =$

```
array([[-1.21621639,  0.50061042,  0.2178842 ],
       [ 1.340521  , -0.06414724, -1.50969118],
       [ 1.24994274,  0.04856202, -1.61096802],
       [ 0.81278559, -0.47179579, -1.28801529],
       [-0.13217938,  0.51304287, -0.48406924]])
```

$h^{l+1} =$

$$h_i^{l+1} = \eta\left(\sum_{j\in\mathcal{N}(i)} A_{ij} W^l h_j^l\right)$$



each row represents node i's aggregated embedding

$$h^{l+1} = \eta($$

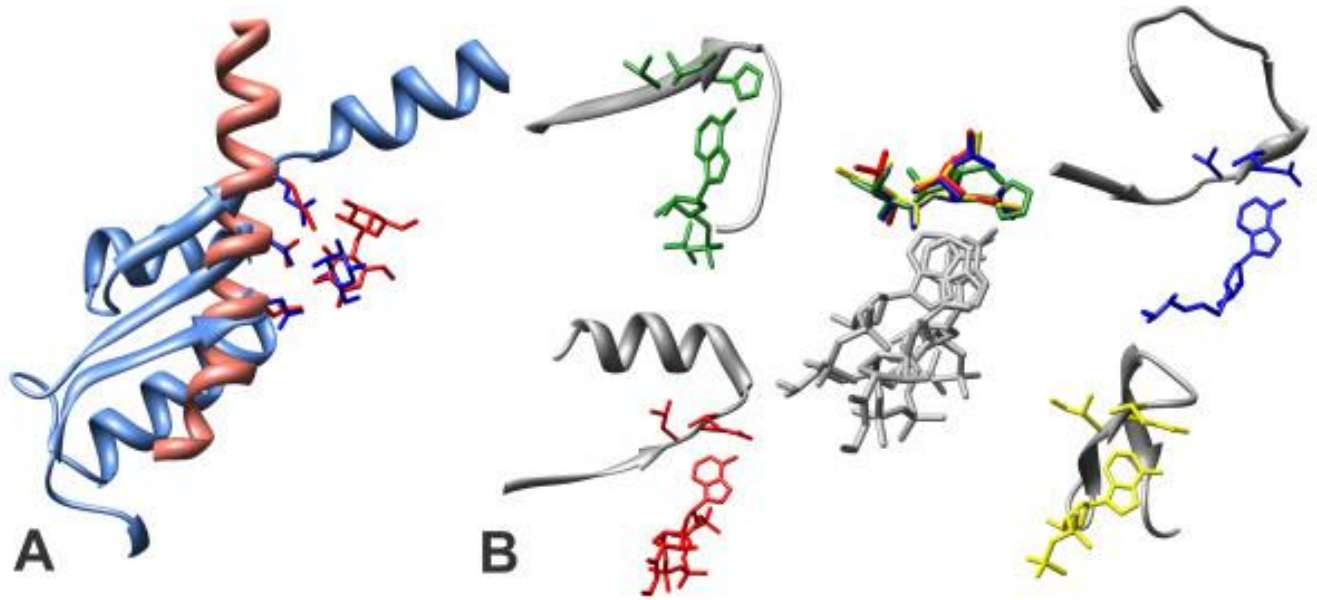
```
array([[ 0.80491082,  0.47771027, -3.06389151],
       [ 1.11776336,  0.56160489, -2.09503726],
       [ 0.03372635,  0.54917244, -1.39308382],
       [ 2.45828436,  0.49745765, -3.60472844],
       [ 0.84651194,  0.07737665, -2.68109912]])
```

what kind of information embedded?

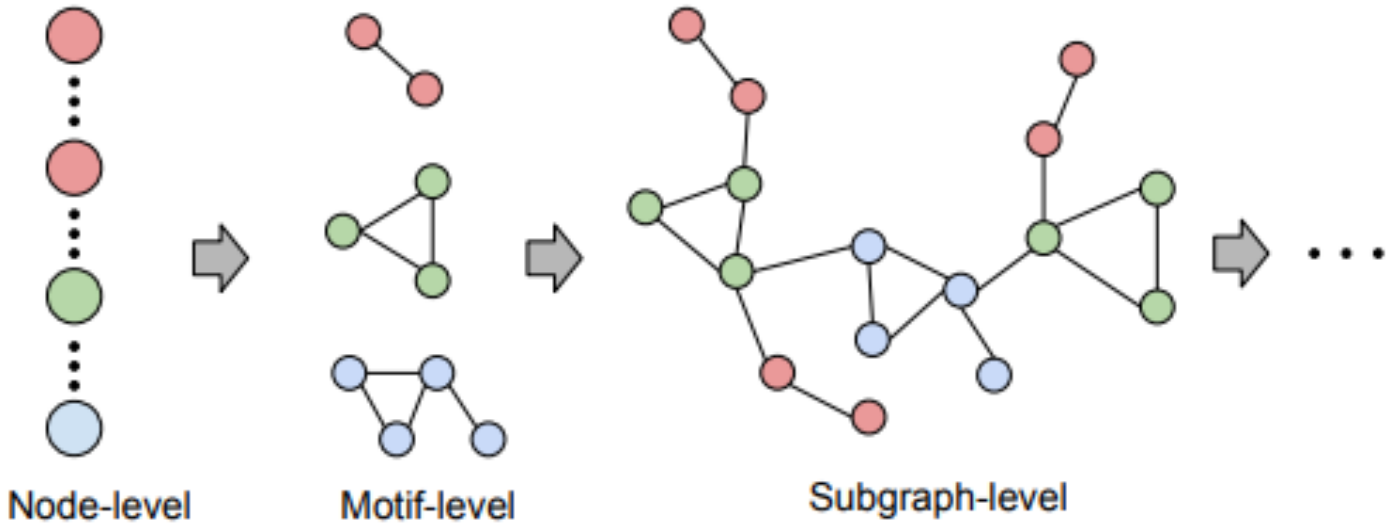
- Structural information such as degree
  - structural motifs
- Nodes' feature information
  - local feature-aggregation like CNN
  - local graph neighborhoods

what kind of information embedded?

- Structural information such as degree
  - structural motifs

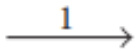
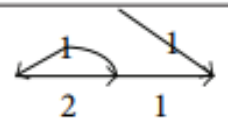
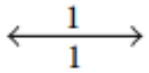
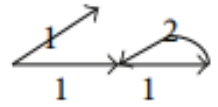
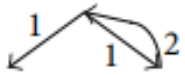
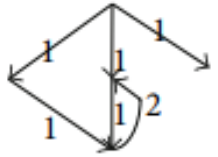
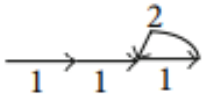
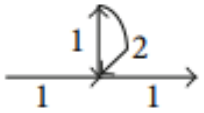
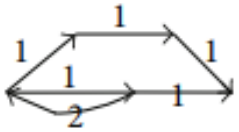


Network Motif



Network Motif (criminal case)

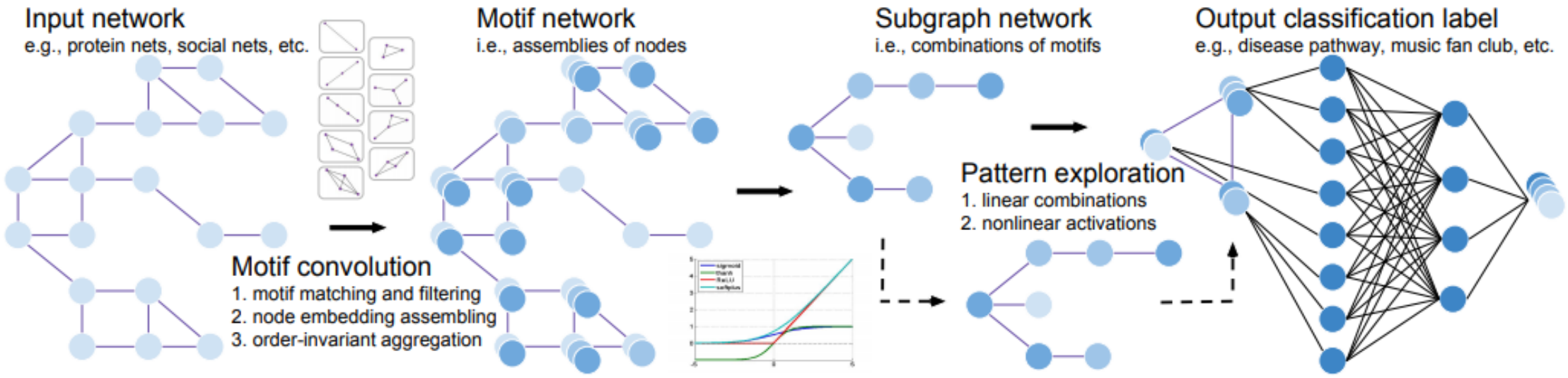
Table 2: The sample of frequent subgraph and network motifs.

Scale	ID	Structure	N_{real}	Z	Scale	ID	Structure	N_{real}	Z
2	S1		230	1.1609	4	M4		30	5.4641
2	S2		10	3.6995	4	M5		30	5.7152
3	M1		15	5.8795	5	M6		136	5.7783
4	M2		38	9.7047	5	M7		126	9.4614
4	M3		36	8.1592	5	M8		102	8.3725

Note of figure: S1-2 are not network motifs ($Z < 5$); M1-8 are network motifs ($Z > 5$), which are structure module with special features (the minimum of Z-score is 5).

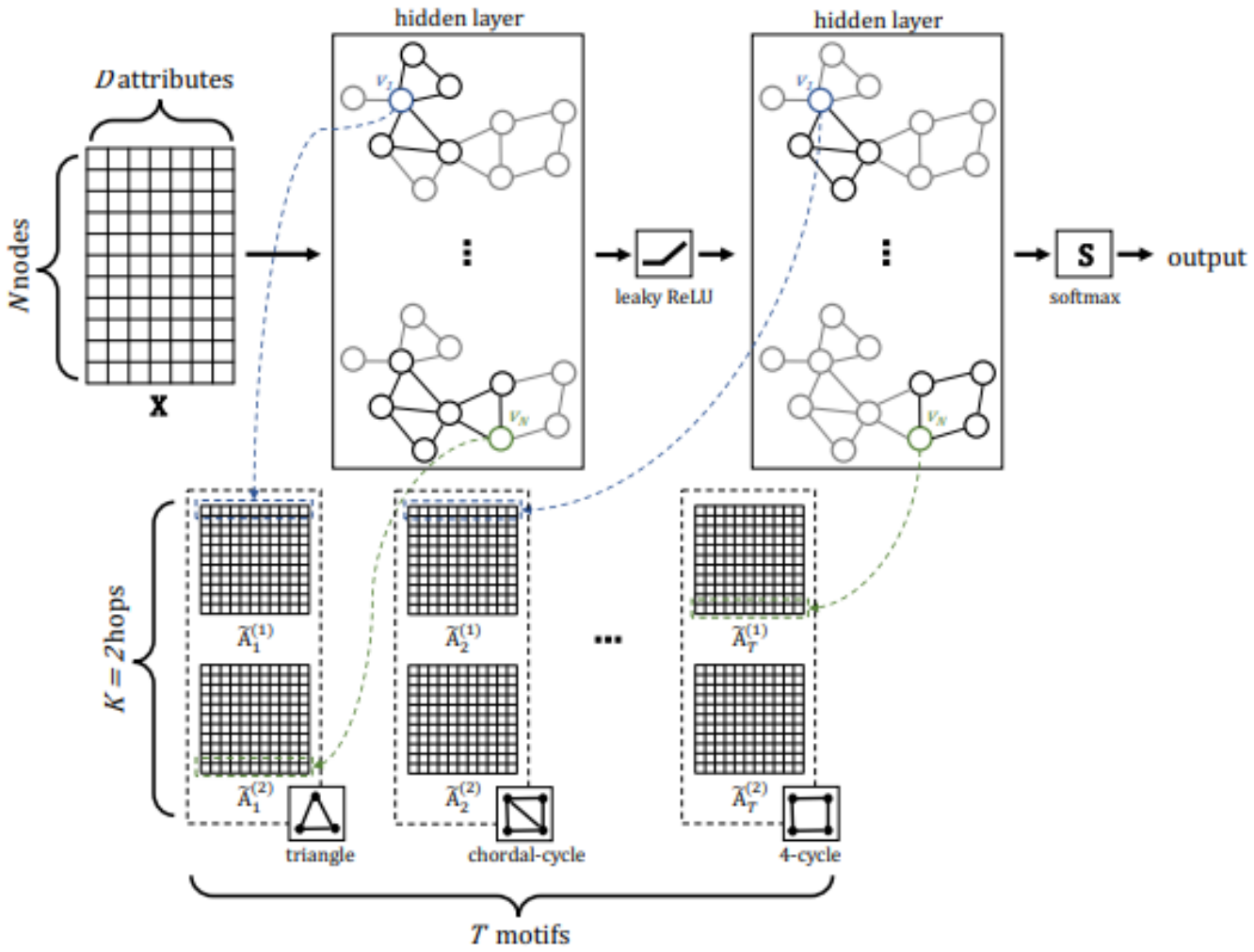
Social Network Analysis Based on Network Motifs. Here

Network Motif with GNN



Node, Motif and Subgraph: Leveraging Network Functional Blocks Through Structural Convolution. [Here]

Network Motif with GNN



Graph Convolutional Networks with Motif-based Attention. Here

Basic GNN

- graph level

$$H^{(l+1)} = \eta\left(AH^{(l)}W^{(l)}_{\text{neigh}} + H^{(l)}W^{(l)}_{\text{self}}\right)$$

- node level

$$h_i^{(l+1)} = \eta\left(W^{(l)}_{\text{self}}h_i^{(l)} + W^{(l)}_{\text{neigh}}\sum_{j\in\mathcal{N}(i)} h_j^{(l)} + \mathbf{b}^{(l)}\right)$$

- η is a activation function such as ReLU

Self-loop GNN

- weight sharing between W_{self} and W_{neigh}

$$H^{(l+1)} = \eta\left((A + I) H^{(l)} W^{(l)} \right)$$

Message passing update

$h_i^{(l)}$:hidden embeddings for node $u \in \mathcal{V}$

$$h_i^{l+1} = \mathrm{UPDATE}\left(h_i^l, \mathrm{AGGREGATE}_{j \in \mathcal{N}(i)} h_j^l\right)$$

$$\equiv \mathrm{UPDATE}\left(h_i^l, m_{j \in \mathcal{N}(i)}^l\right)$$

where $\mathrm{AGGREGATE}$ and UPDATE are differentiable functions (neural network),

$m_{j \in \mathcal{N}(i)}^l$ is ***message*** on neighborhood j

Neighborhodd Aggregation: normalization

$$m_{\mathcal{N}(i)} = \frac{\sum_{j \in \mathcal{N}(i)} h_j}{d_i}$$

$$OR$$

$$m_{\mathcal{N}(i)} = \frac{\sum_{j \in \mathcal{N}(i)} h_j}{\sqrt{d_i d_j}} \quad (1)$$

GNN : Why normalize?

- Simple summation would cause numerical instabilities and difficulties for optimization due to node degrees
- Kipf and Welling(2016) argue that information from very high-degree nodes may not be useful for inferring community membership in the graph
- Basic GNN with symmetric normalization results in a first-order approximation of a spectral graph convolution.

GNN : normalization's pros and cons

- **pros**: numerical or optimzation stability
- **cons**: various structural graph feasures, such as, degrees, can be obscured
- normalization should depend **on application-specific** matter

GNN: GraphSage

GraphSage

- Hamilton (2017): Generalized neighborhodd aggregation
  - differentiate weights b/w neighbors' and central node(s)

$$\underset{d\times 1}{h_i^{(l+1)}} = \eta\left(\overset{d\times d}{W_{\text{self}}^{(l)}} h_i^{(l)} + m_{\mathcal{N}(i)}\right)$$

$$m_{\mathcal{N}(i)} = \begin{cases} \frac{1}{d_i}\sum_{j\in\mathcal{N}(i)} W_{\text{neigh}}^{(l)} h_j^{(l)} \\ \text{Max}_{j\in\mathcal{N}(i)}\left(W_{\text{neigh}}^{(l)} h_j^{(l)}\right) \\ \text{LSTM}_{j\in\mathcal{N}(i)}\left(h_j^{(l)}\right) \end{cases}$$

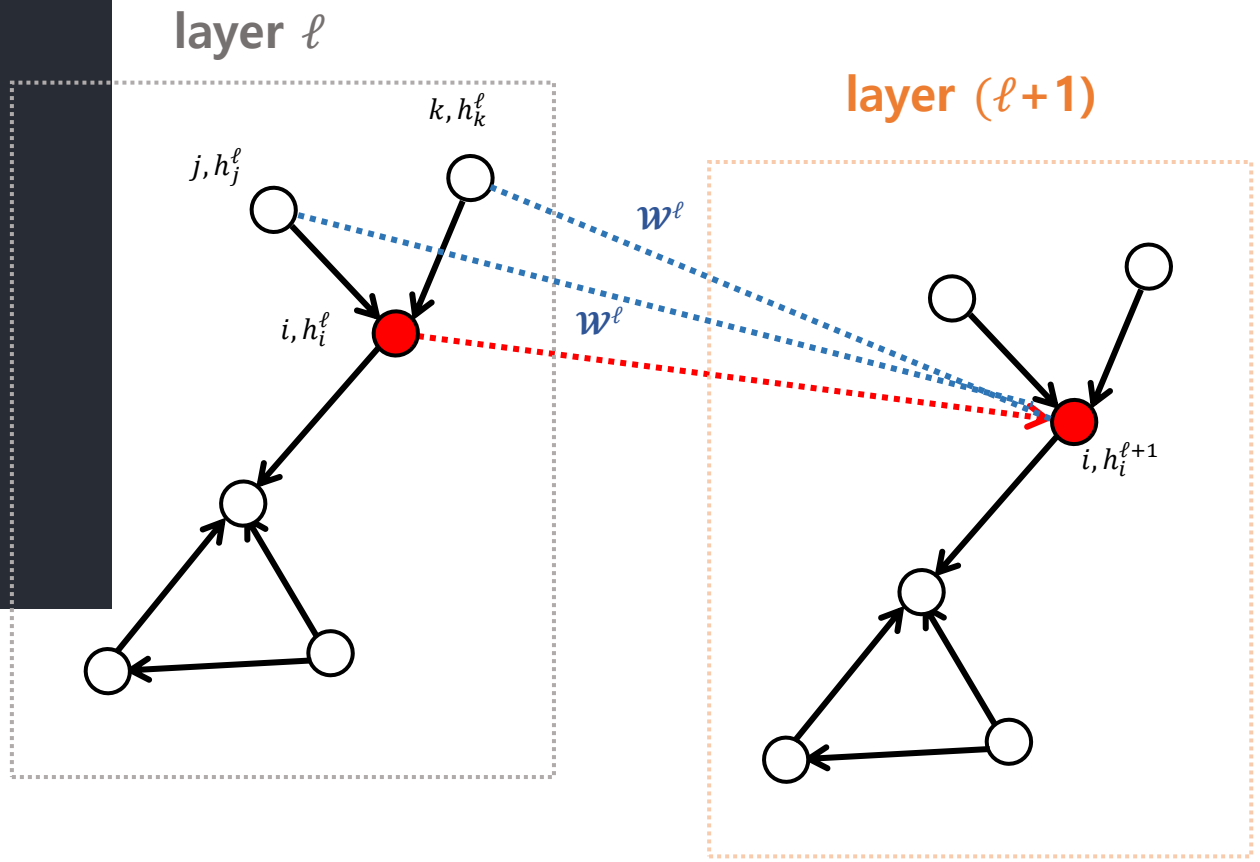
GNN: GraphSage

GraphSage

- Hamilton (2017): Generalized neighborhodd aggregation
  - differentiate weights b/w neighbors' and central node(s)

$$h_i^{(l+1)}_{d\times 1} = \eta\left(W_{\text{self}}^{(l)}{}^{d\times d} h_i^{(l)} + m_{\mathcal{N}(i)}\right)$$

$$m_{\mathcal{N}(i)} = \begin{cases} \frac{1}{d_i}\sum_{j\in\mathcal{N}(i)} W_{\text{neigh}}^{(l)} h_j^{(l)} \\ \text{Max}_{j\in\mathcal{N}(i)}\left(W_{\text{neigh}}^{(l)} h_j^{(l)}\right) \\ \text{LSTM}_{j\in\mathcal{N}(i)}\left(h_j^{(l)}\right) \end{cases}$$



GNN : GCN

Graph Convolution Networks (GCNs)

- The most popular GNN models outlined by Kipf and Welling(2016)
- graph level

$$h^{(l+1)} = \eta\left(D^{-1}Ah^{(l)}W^{l}\right)$$

$$h_i^{(l+1)} = \eta\left(\frac{1}{d_i}\sum_{j\in\mathcal{N}(i)} A_{ij}W^{l}h_j^{(l)}\right)$$

GNN : GCN

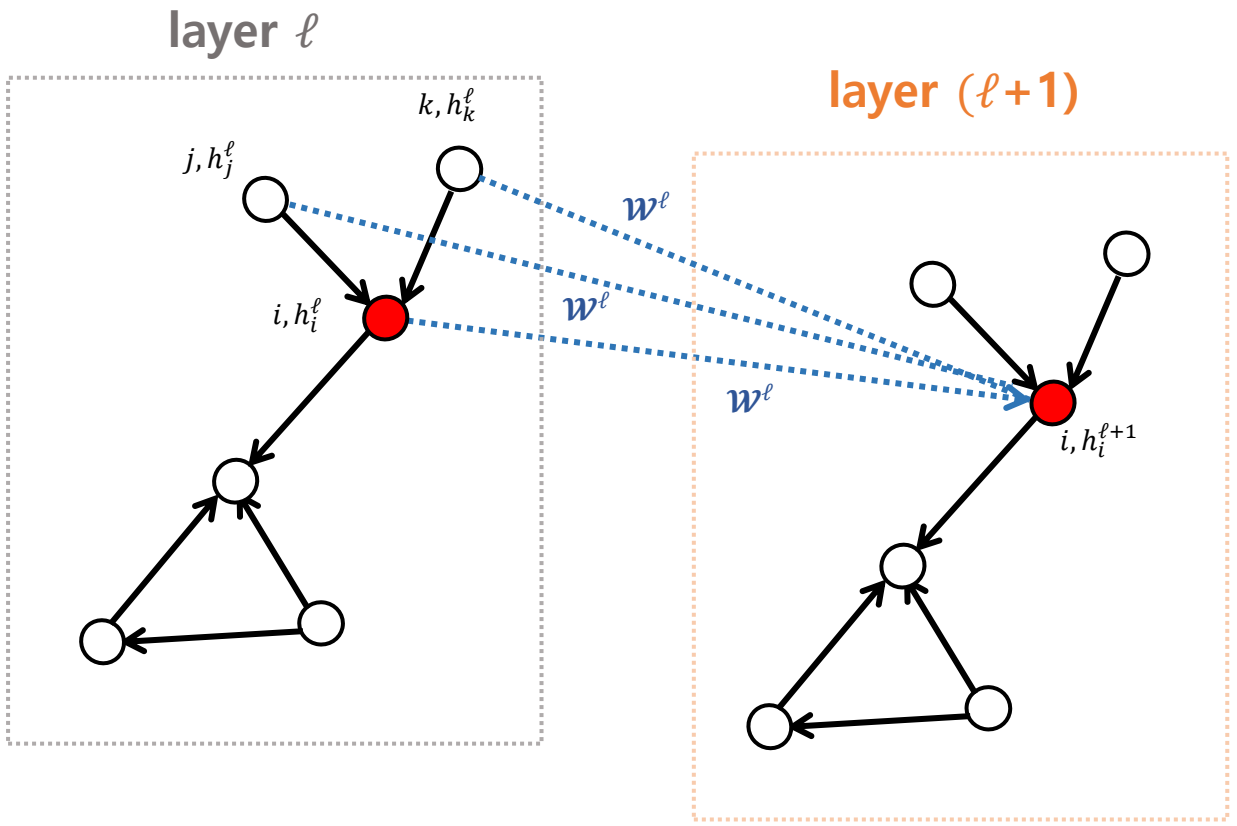
$$\underset{(d\times d)}{h^{(l+1)}} = \eta\left(\underset{(n\times n)}{D^{-1}} A h^{(l)} W^{l}\right)$$

$$h_i^{(l+1)} = \eta\left(\frac{1}{d_i}\sum_{j\in\mathcal{N}(i)} A_{ij} W^{l} h_j^{(l)}\right)$$

GNN : GCN

$$\underset{(d\times d)}{h^{(l+1)}} = \eta\left(\underset{(n\times n)}{D^{-1}} A h^{(l)} W^{l}\right)$$

$$h_i^{(l+1)} = \eta\left(\frac{1}{d_i}\sum_{j\in\mathcal{N}(i)} A_{ij} W^{l} h_j^{(l)}\right)$$



Graph Attention Model (GAT)

$$m_{j\in\mathcal{N}(i)} = \sum_{j\in\mathcal{N}(i)} \alpha_{ij} h_j$$

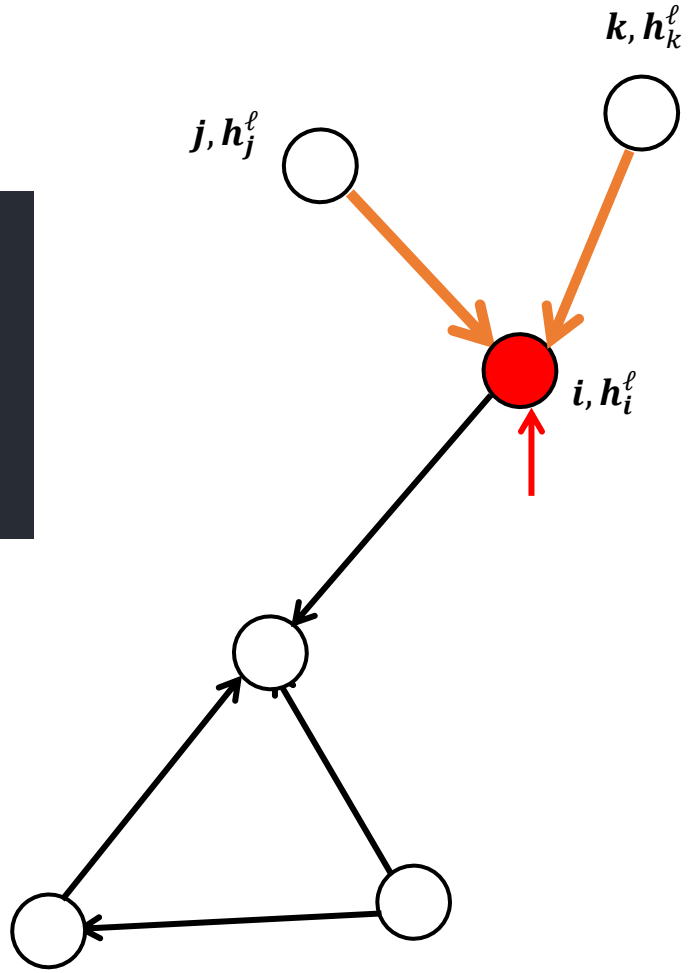
$$\alpha_{ij} = \begin{cases} \dfrac{\exp\left(a^T\left[Wh_i \| Wh_j\right]\right)}{\sum_{k\in\mathcal{N}(i)} \exp\left(a^T\left[Wh_i \| Wh_k\right]\right)} \\ \dfrac{\exp\left(h_i^T W h_j^T\right)}{\sum_{k\in\mathcal{N}(i)} \exp\left(h_i^T W h_k^T\right)} \\ \dfrac{\exp\left(\mathrm{MLP}(h_i, h_j)\right)}{\sum_{k\in\mathcal{N}(i)} \exp\left(\mathrm{MLP}(h_i, h_k)\right)} \end{cases}$$

$$\Longrightarrow$$

$$\underset{(n\times d)}{h^{(l+1)}} = \eta\Big(\underset{(n\times n)}{A}\ \underset{(n\times d)}{\alpha}\ \underset{(d\times d)}{W^{(l)}}\ h^{(l)}\Big), \quad \underset{(d\times 1)}{h_i^{(l+1)}} = \eta\Big(\sum_{j\in\mathcal{N}(i)} \overset{\text{scalar}}{\alpha_{ij}^{(l)}}\ \underset{(d\times d)}{W^{(l)}}\ h_j^{(l)}\Big)$$

GNN : Graph Attention

$$\underset{(n\times d)}{h^{(l+1)}} = \eta\Big(\underset{(n\times n)}{A}\ \underset{(n\times d)}{\alpha}\ \underset{(d\times d)}{W^{(l)}}\ h^{(l)}\Big),\quad \underset{(d\times 1)}{h_i^{(l+1)}} = \eta\Big(\sum_{j\in\mathcal{N}(i)} \overset{\text{scalar}}{\alpha_{ij}^{(l)}}\ \underset{(d\times d)}{W^{(l)}}\ h_j^{(l)}\Big)$$



GNN : Graph Attention

- What are a_{ij}?
  - it measures the node $j's$ message importance to i?
  - By using softmax normalization due to direct comparison of each node's message across different neighborhoods.

Graph Transformers

Transformers

- Special case of GCNs when the graph is ***fully connected***
  - neighbor $\mathcal{N}(i)$ is the whole graph

- Matrix notation

$$\underset{(d\times 1)}{h^{(l+1)}} = \text{Concat}_{k=1}^{K}\left(\text{softmax}\left(Q^{(l)}\ K^{(l)^T}\cdot V^{(l)}\right)\right)W^{(l)}$$

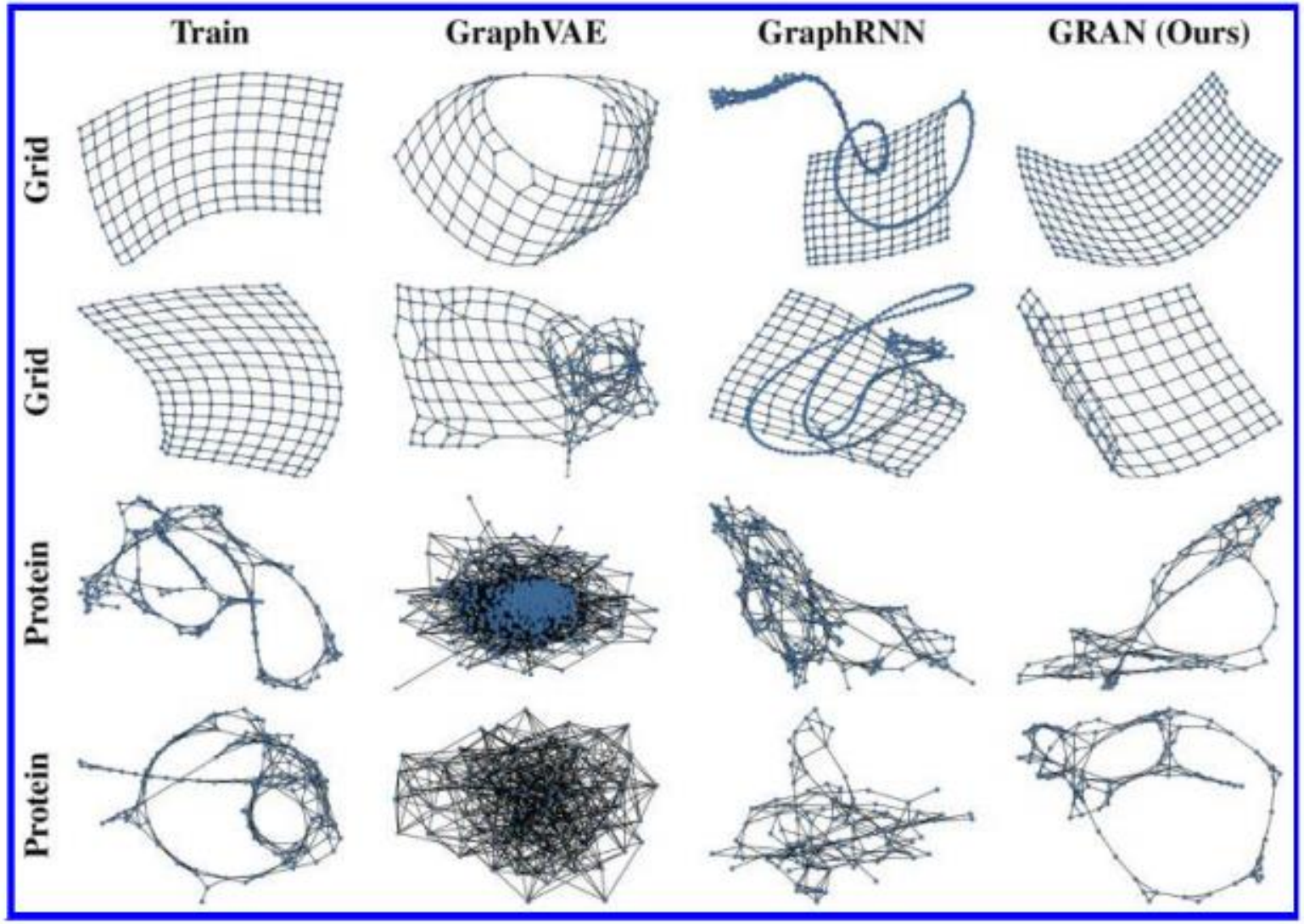
$$\underset{(n\times \frac{d}{K})}{Q^{(l)}} = h^{(l)}\ W_Q^{(l)}\ \ (\text{Query})$$

$$K^{(l)} = \underset{(d\times 1)}{h^{(l)}}\ W_K^{(l)}\ \ (\text{Key})$$

$$V^{(l)} = h^{(l)}\ \underset{(d\times \frac{d}{K})}{W_V^{(l)}}\ \ (\text{Value})$$

https://arxiv.org/pdf/2107.07999.pdf
Graph Kernel Attention Transformers

Various Generative models



Next

- More GNN models
- Generative models
  - GRAPH + VAE
  - GRAPH + Generative models